

WestCV — план решения

Что мы выяснили, архитектура моделей, разметка данных, датасеты и план работ

Сводный документ для команды из трёх человек. Собран из PDF задания, кода стартового кита организаторов, метаданных сэмплов и двух наших отчётов:

`WIUT_CV_Track_analysis_RU.md` и `WIUT_toolkit_RU.md`.

14

официальных классов событий (Part A)

4 сэмпла

≈18.4 мин, ≈20 ГБ, без разметки

4K · 29.97p

H.264 High 4:2:2 10-bit, ~140 Мбит/с

≤ 3×

длительности видео на Part A + Part B вместе

≤ 5 ГБ

весов; офлайн, T4 16 ГБ, 8 CPU, 32 ГБ RAM

Что внутри

- Проверенные факты о сэмплах, харнесе и метрике, которых нет в PDF.
- Архитектура Part A и Part B: детектор, трекер, геометрия сцены, правила, верификатор аварий, риск-модель.
- Как будет выглядеть разметка: инструменты, конфиги, форматы, гайдлайн, протокол согласия.
- Все датасеты со ссылками, лицензиями и ролью в проекте.
- План по вехам, роли, риски и вопросы организаторам.

Состояние на 25.09.2026

- Анализ задания и стратегия готовы (51 исправление после финальной проверки по коду кита).
- Каталог инструментов, моделей и датасетов готов; конвертеры разметки лежат в `tools/annotation/`.
- Решение ещё не реализовано: `solution.py` — шаблон организаторов.
- Следующий шаг — день D0: сэмплы, прокси, бенчмарк декодирования на T4, тег `v0.1`.

Обозначения. (оценка) — число получено расчётом или экстраполяцией, не измерено на целевом железе; **(не подтверждено)** — первоисточник проверить не удалось; **[орг.]** — нужен ответ организаторов (раздел 10); **[camera.md]** — зависит от описания сцены, которого пока нет. Все пороги задаются в секундах и метрах, никогда в кадрах. Документ не является юридической консультацией по лицензиям.

Содержание

1. Главное на одной странице	4
2. Задача и как её оценивают	5
2.1 Что сдаём и интерфейс	5
2.2 Формулы оценки и цена каждой составляющей	5
2.3 Ограничения и рубрики	6
2.4 Как харнесс обрабатывает видео (по коду)	7
3. Что мы выяснили	8
3.1 Сэмплы и тестовый набор	8
3.2 Бюджет времени: декодирование решает всё	8
3.3 Метрика Part A: что из неё следует	9
3.4 Метрика Part B: механика и ловушки	10
3.5 Открытые развилки	12
4. Архитектура решения	13
4.1 Общая схема Part A	13
4.2 Декодирование и бюджет-планировщик	13
4.3 Детектор	14
4.4 Трекер и офлайн-ремонт треков	15
4.5 Геометрия сцены и регистрация	16
4.6 Общие библиотеки: следы, полосы, кинематика, конфликты	17
4.7 Светофор, огонь и дым, препятствия	17
4.8 Аварии и near-miss: кинематика + верификатор	18
4.9 Part B: причинный RiskEstimator	20
4.10 Постобработка сегментов	21
4.11 Итоговый стек и бюджет весов	21
4.12 Инженерия: установка, офлайн, детерминизм, репозиторий	22
5. Все 14 классов: конвенции и методы	24
5.1 Официальные определения и границы	24
5.2 Как мы детектируем каждый класс	24
5.3 Какие классы включаем	26
6. Разметка данных	27
6.1 Три задачи разметки и инструменты	27
6.2 Подготовка видео: прокси с сохранением кадров	27
6.3 События в Label Studio: как это выглядит	28
6.4 Формат результата: ground_truth.json	30
6.5 Гайдлайн по границам	30
6.6 Двойная разметка, согласие, арбитраж	31
6.7 Боксы для дообучения детектора	32
6.8 Геометрия сцены	32
6.9 Проху-набор аварий, синтетика, гигиена данных	33
7. Датасеты: ссылки, лицензии, роль	34
7.1 Правила работы с внешними данными	34
7.2 Основные датасеты (топ-8 + COCO)	34
7.3 Запасные и опциональные	35
7.4 Проверенные и отклонённые	36
7.5 План загрузки и обучающие смеси	37

7.6 Лицензии и развилка AGPL против NC-SA	37
8. План работ	39
8.1 Роли	39
8.2 Вехи и критерии выхода	39
8.3 Ближайшие шаги	40
8.4 Сайт и live-демо	40
9. Риски	42
10. Вопросы организаторам	43
11. Ссылки	44

Об этом документе

Документ собран 25.09.2026 из материалов репозитория `asanashirov/westcv`. Поля «(не подтверждено)» и оценки скорости без ссылки на таблицы авторов перепроверить до того, как на них опираться. Если организаторы опубликуют определения классов, пример GT, camera.md или формат тестовых видео — они важнее этого документа. Исходник: `docs/plan/WestCV_plan_RU.html`, сборка: `python docs/plan/build.py`.

1. Главное на одной странице

- 1 Главный риск — не качество модели, а запуск и время.** Если пакет не запустился, модельная оценка M обнуляется: в типичном сценарии это -0.26 отбора (26 баллов из 100). Для сравнения, всё live-демо стоит 7.5 балла.
- 2 Сэмплы тяжелее, чем обещает PDF:** 3840×2160 , 29.97p (в PDF — «25 fps»), H.264 High 4:2:2 10-bit, ~140 Мбит/с. Один только проход харнесса `cv2.read()` для Part B занимает $0.84\text{--}1.0 \times$ длительности на 8 vCPU под Linux и $\approx 1.4\text{--}2.1 \times$ на AWS g4dn.2xlarge (оценка). Лимит — $3 \times$ на Part A и Part B вместе. NVDEC на T4 этот формат не декодирует.
- 3 Один закон бюджета:** $T_A = 3 \cdot \text{dur} - 1.2 \cdot (\text{Part B вместе с декодом харнесса}) - 0.2 \cdot \text{dur}$. План строится из `meta` и таблицы стоимостей, измеренной на T4; настенные часы — только аварийный тормоз. Модели грузим и прогреваем при импорте `solution.py` — это время в бюджет видео не входит.
- 4 Score A — macro-F1 по множеству C = (классы GT) и (наши классы).** Класс, которого нет в тесте, но который мы хоть раз предсказали, добавляет в среднее ноль. Предсказываем класс, только если $p \cdot f > (1-p) \cdot q \cdot \tilde{f}$. Стартовый набор: `stopped_vehicle`, `jaywalking`, `wrong_way`, `congestion`, `accident`.
- 5 F1 при tIoU 0.7 определяется точностью границ.** Допустимый сдвиг для события 2 с — 0.35 с, для 5 с — 0.88 с. Границы привязываем к геометрическим моментам (касание, пересечение стоп-линии, вход в полигон), а для классов с окном подтверждения переносим старт назад.
- 6 Постобработка стоит больше, чем подбор моделей:** интервалы по трекам → объединение → склейка разрывов → минимальная длительность → санитайзер (длина ≥ 0.05 с, округление до 3 знаков, конец $\leq \text{floor}(\text{duration}, 3)$). Событие короче 1 мс после округления заставит `evaluate.py` отвергнуть весь файл.
- 7 Архитектура Part A:** единый CPU-декод → YOLO26m (дообученный на наших кадрах) → ByteTrack → следы в метрах через гомографию → модули правил на геометрии `scene.yaml` → уточнение границ → постобработка. Обучаются детектор, детектор огня и дыма и (опционально) кроп-верификатор аварий; всё остальное — правила.
- 8 accident и near_miss строятся на парной кинематике:** зазор между следами, TTC/PET, Δv за 1 с, hinge-fit момента начала. Верификатор X3D или VideoMAEv2 только подтверждает кандидата и никогда не задаёт границы. VLM по умолчанию выключен.
- 9 Part B весит 18 баллов, но только если в тесте есть аварии.** Собственная причинная перцепция, никаких выходов Part A. Сигнал двухканальный: ранжирующий канал ≤ 0.4999 , тревога ≥ 0.5 только при уверенном неизбежном конфликте. Тревоги короткие (≤ 8 с), бюджет ложных — ≤ 1.8 в час.
- 10 Разметка:** Label Studio (TimelineLabels, `frameRate = 29.97002997`) на 1080p-прокси, где сохранён каждый кадр. Каждый сэмпл размечают двое независимо, согласие меряем официальным `evaluate.py` (цель $F1 \geq 0.8$ при $\tau = 0.5$). Боксы — X-AnyLabeling, геометрия — labelme.
- 11 Датасеты:** ACCIDENT, TAD, AI City 2021 Track 4, UA-DETRAC, D-Fire, RAOD, MIO-TCD, CADP, плюс COCO и Street Scene. До обучения нужно решить развилку «веса Ultralytics под AGPL-3.0 против данных под NC-SA».
- 12 Сайт даёт 25% отбора, одно live-демо — 7.5 балла** (эквивалент $+0.18$ к Score A). Статический сайт на GitHub Pages плюс Gradio-бэкенд на HF Space.
- 13 Порядок работ: сначала рабочая сдача, потом улучшения.** D0 — сэмплы, прокси, бенчмарк декодирования, `ver0.1`. Вопросы организаторам (раздел 10) отправить сразу: главные — формат тестовых видео, CPU оценочной машины, дедлайн.

2. Задача и как её оценивают

Фиксированная CCTV-камера смотрит на дорогу. Для каждого `.mp4` нужно:

- **Part A (обязательно)** — вернуть все события как список `[start_sec, end_sec, label]` по 14 официальным классам. Видео можно читать как угодно: несколько проходов, произвольный доступ, взгляд в будущее.
- **Part B (бонус)** — на каждом кадре вернуть вероятность того, что авария *начнётся* в ближайшие 5 с, используя только уже полученные кадры.

2.1 Что сдаём и интерфейс

1. **Публичный Git-репозиторий на tere.** Работает на чистой машине двумя командами, веса лежат в пакете или скачиваются `weights/download.sh` один раз до оценки.
2. **Публичный сайт:** команда, подход, EDA сэмплов, визуализации результатов, live-демо с загрузкой видео, отчёт, ссылки.
3. **Короткий технический отчёт** (может быть страницей сайта): что сделали, что сработало, что нет.

```
pip install -r requirements.txt # или: docker build -t team .
python run_submission.py --videos /data/test --out predictions.json
```

Интерфейс `solution.py` (имена и сигнатуры менять нельзя; классы можно только удалять):

```
CLASSES = ["accident", "near_miss", "red_light", "wrong_way", "illegal_u_turn",
            "stopped_vehicle", "jaywalking", "failure_to_yield", "illegal_turn",
            "solid_line_crossing", "stop_line", "congestion", "road_obstacle", "fire_smoke"]

def detect_events(video_path: str) -> list[list]: # Part A
    """[[start_sec, end_sec, label], ...]; 0 <= start < end <= duration"""

class RiskEstimator: # Part B, причинный
    def reset(self, meta: dict) -> None: ... # meta: video_id, fps, width, height, n_frames
    def step(self, frame: np.ndarray, t_sec: float) -> float: ... # BGR uint8 (H, W, 3) -> [0, 1]
```

`predictions.json` (пишет харнесс):

```
{
  "team": "westcv",
  "videos": {
    "test_001.mp4": {
      "events": [[12.4, 18.9, "accident"],
                 [40.0, 43.5, "red_light"]],
      "risk": [[0.00, 0.01], [0.04, 0.01]]},
    "test_002.mp4": {"events": [], "risk": []}}
```

`ground_truth.json` (скрытый; такой же делаем для dev):

```
{
  "test_001.mp4": {
    "duration": 600.0, "fps": 25.0,
    "events": [[12.0, 19.0, "accident"]]]}
```

Ключи — имена файлов, не пути. Каждое тестовое видео должно присутствовать, даже с пустым списком.

2.2 Формулы оценки и цена каждой составляющей

$\text{Score A} = (1/|C|) \cdot \sum_{c \in C} (1/3) \cdot \sum_{\tau \in \{0.3, 0.5, 0.7\}} F1_c(\tau)$, C = классы GT и классы наших предсказаний
 $\text{Score B} = 0.4 \cdot \text{AP} + 0.4 \cdot F1_{\text{alarm}} + 0.2 \cdot \min(1, \text{mTTA} / W)$, $H = 5$ с, $W = 10$ с, $\theta = 0.5$
 $M = 0.7 \cdot \text{Score A} + 0.3 \cdot \text{Score B}$ ($M = \text{Score A}$, если в тесте нет аварий)
 $\text{Отбор} = 0.6 \cdot M + 0.25 \cdot \text{Website} + 0.15 \cdot \text{Code} = 0.42 \cdot A + 0.18 \cdot B + 0.25 \cdot \text{Web} + 0.15 \cdot \text{Code}$

Сопоставление в Score A: для каждого класса и τ считается temporal IoU всех пар «GT-предсказание» одного видео, пары сопоставляются жадно по убыванию IoU (пара засчитывается, если $\text{IoU} \geq \tau$ и оба сегмента свободны). TP, FP и FN суммируются по всем видео, потом считается F1. Векторизованная копия нашего скорера совпала с официальным Score B с точностью $2.2 \cdot 10^{-16}$ на 40 случайных случаях.

Составляющая	Баллы отбора (из 1.0)	Эквивалент в Score A
Score A целиком	0.42 (0.60 без аварий)	—
+0.10 к Score A	0.042	0.10
Score B целиком	0.18 (0 без аварий)	—

Составляющая	Баллы отбора (из 1.0)	Эквивалент в Score A
+0.10 к Score B	0.018	0.043
Сайт: live-демо (30%)	0.075	0.18
Сайт: визуализации сэмплов (20%)	0.050	0.12
Сайт: EDA (15%) / подход и отчёт (15%)	0.0375 / 0.0375	0.09 / 0.09
Сайт: команда (10%) / дизайн, UX, extras (10%)	0.025 / 0.025	0.06 / 0.06
Code: запускается как сдано (40%)	0.060	0.14
Code: воспроизводимость (25%) / структура (20%) / инженерные решения (15%)	0.0375 / 0.030 / 0.0225	0.09 / 0.07 / 0.05

Сценарий A = 0.40, B = 0.20, Web = 0.8, Code = 0.85

Итог 0.532. Если пакет не запустился — 0.268: потеря 0.264, и это нижняя оценка, потому что вместе с запуском, скорее всего, пропадают баллы за воспроизводимость. Полный отказ от Part B стоит всего 0.02–0.05. Выводы: надёжный запуск и сайт — приоритет, а не «доделать в конце»; страница команды даёт 2.5 балла примерно за 1.5 часа; демо, которое не падает, стоит примерно 40% идеального Part B.

Направление	Доля ценности (оценка)	Доля инженерных усилий
Part A: детектор, трекер, геометрия, правила, постобработка	≈42% отбора (60% без аварий)	≈45%
Сайт: демо, визуализации, EDA, отчёт, команда	25%	≈25–30%
Упаковка, воспроизводимость, бюджет	15% (плюс защита всего M)	≈15%
Part B	≤ 18%, условно	≈10–15% (≤ 1 человеко-недели)

2.3 Ограничения и рубрики

Пункт	Ограничение
Машина оценки	1× NVIDIA T4-класса (16 ГБ VRAM), 8 ядер CPU, 32 ГБ RAM. Организаторы могут изменить до дедлайна.
Интернет при оценке	Нет. Все веса — в пакете или в архиве, который скачивается до прогона.
Время на видео	Part A + Part B вместе ≤ 3× длительности видео (wall-clock). Превышение — видео засчитывается пустым, включая события Part A .
Веса	≤ 5 ГБ суммарно.
Python и зависимости	3.10 или новее; requirements.txt или Dockerfile.
Модели	Только open-weights, которые можно положить в пакет. Никаких OpenAI, Gemini, Anthropic и других хостинговых моделей на инференсе. Для кода, сайта и отчёта — можно.
Данные	Публичные датасеты и своя разметка сэмплов. Нельзя собирать записи с той же камеры другими путями; найденные — не использовать.
Интерфейс	run_submission.py и evaluate.py не менять. RiskEstimator.step видит только полученные кадры: чтение файла внутри или выходы Part A, посчитанные по будущим кадрам, — нарушение.
Детерминизм	Фиксированные сиды; два прогона на одной машине дают одинаковый predictions.json с точностью до float-шума.
Сдача	Оценивается тегированный коммит на момент дедлайна. Нарушение правил — дисквалификация.

Рубрика Code	Вес	Рубрика Website	Вес
Запускается как сдано: обе команды на чистой машине, без ручных шагов	40%	Live-демо: загрузка работает, результат с визуализацией, без падений	30%
Воспроизводимость: веса, сиды, датасеты, скрипты обучения; результат совпадает с predictions_samples.json	25%	Визуализации сэмплов: все видео размечены, шкалы и кривые риска читаемы	20%
Структура: понятные модули, нет мёртвого кода, ноутбук не единственный источник	20%	EDA: глубже подсчёта кадров, находки, повлиявшие на решение	15%
Инженерные решения: сэмплирование кадров, батчи, кэш, запас по времени	15%	Подход и отчёт: по странице можно пересобрать конвейер	15%
		Команда и портфолио	10%
		Дизайн, UX, extras: быстро, работает на телефоне	10%

Пакет, который не запустился после одной попытки поправить очевидную проблему окружения, получает model score 0.

2.4 Как харнесс обрабатывает видео (по коду)

```
импорт solution.py (load_solution): загрузка моделей — ДО любых таймеров, не тарифицируется
для каждого видео по очереди:
    t0 = perf_counter(); budget = 3 * CAP_PROP_FRAME_COUNT / CAP_PROP_FPS
    events = detect_events(path) # Part A; читает файл как угодно; никто её не прерывает
    если Part A уже вышла за бюджет -> Part B пропускается
    est = sol.RiskEstimator(); est.reset(meta) # новый объект на каждое видео, уже внутри таймера
    cv2.VideoCapture(path): read() КАЖДОГО кадра -> step(frame, idx/fps) # дедлайн проверяется раз в 100
    кадров
    если total > budget -> events = [], risk = [] (Part A тоже теряется!)
    исключение в detect_events -> events = []
    исключение внутри step -> risk = [] для всего видео, events сохраняются
    step вернул не скаляр / None -> 0.0 для этого кадра; NaN -> 0
    predictions.json пишется один раз, после BCEX видео (segfault или OOM-kill убивает весь прогон)
```

Поведение (проверено по коду кита)	Наше решение
load_solution() вызывается до цикла по видео и до t0	Грузим все модели, контекст CUDA, cuDNN и делаем прогон-прогрев при импорте
RiskEstimator() создаётся заново на каждое видео, внутри таймера	Модели — синглтоны уровня модуля; в reset() только состояние видео
t = idx/fps, fps = CAP_PROP_FPS; duration = CAP_PROP_FRAME_COUNT/fps	Та же база времени везде: Part A, разметка, сайт
clean_events: float()/str(), end ограничивается duration, проверка 0 ≤ s < e ДО округления, затем округление до 3 знаков	Свой санитайзер: длина ≥ 0.05 с → округление → end ≤ floor(duration, 3)
Один класс: сортировка по (label, start); сегмент с s < last_end выбрасывается , а не сливается. Касание [5,8]+[8,9] допустимо	Перекрытия объединяем сами до выдачи
Риск пишется как round(clip(score), 4)	Вне тревоги не отдавать ≥ 0.49995 (превратится в 0.5); всегда Python float
Нет signal / kill; Part A не прерывается никогда	Собственный watchdog внутри detect_events — единственная защита
evaluate.py сначала проверяет формат: любая ошибка → exit 1; допуск end ≤ duration + 0.5	В CI: evaluate.py --validate-only --gt dev_gt.json (без --gt длительности и наличие всех видео не проверяются)
Исключение в step → risk = [], события сохраняются; PDF называет это «пустым предсказанием для видео»	Расхождение кода и PDF — вопрос к организаторам [орг.]; на исключение как на стратегию не рассчитываем
В predictions.json есть блок log и поле team (по умолчанию unnamed-team)	При сравнении двух прогонов log игнорировать; передавать --team westcv

3. Что мы выяснили

Этот раздел — факты, которых нет в PDF задания: метаданные сэмплов, замеры декодирования, поведение харнесса и следствия из метрики, проверенные на официальном скорере.

3.1 Сэмплы и тестовый набор

Файл	Кадров	Длительность	Съёмка (часы камеры)	Замечание
C3896.MP4	10 200	340.3 с	18.09.2026, 12:18	Дневная сессия; кандидат в эталон сцены
C3897.MP4	9 525	317.8 с	18.09.2026, 12:24	Дневная сессия
C3902.MP4	9 525	317.8 с	18.09.2026, ~18:00	Вечер. Размер совпадает с C3897 до байта — сверить хеши, возможен дубликат
C3905.MP4	3 825	127.6 с	18.09.2026, до 18:24	Вечер, солнце низко (закат в Ташкенте ≈18:28, оценка)

Формат всех файлов: Sony ILCE-6700, 3840×2160, 29.97p (30000/1001), H.264 High 4:2:2 10-bit (AVC140_3840_2160_H422P@L51), ~140 Мбит/с, GOP 30, гамма rec709, звук LPCM 48 кГц стерео. Всего ≈18.4 мин и ≈20.3 ГБ.

Ссылки на сэмплы (из Videos.pdf; соответствие ссылок именам файлов в PDF не указано):

- <https://drive.google.com/file/d/1kR9jODA2Wotw4gwkvprKdqFADNJNc1nS/view>
- https://drive.google.com/file/d/1hp8DYeqtYHSwfM6qAo9FPSRHlpMFrIN_/view
- <https://drive.google.com/file/d/10cHEReCWzO3u-Vk1CnNgHAX6egGy5MwJ/view>
- <https://drive.google.com/file/d/1aJ-QsAZVYJtLKHiRvKKeBq1D3GWNobRd/view>

Google Drive отдаёт «Quota exceeded»

Это лимит скачиваний конкретного публичного файла. Надёжный обход — копирование на стороне сервера в свой Drive:

- **Colab:** `auth.authenticate_user()`, `Drive API files().copy(...)`, затем `MediaIoBaseDownload` фрагментами по 256 МБ и сверка `md5Checksum`. Если не хватает прав — вручную «Создать копию» в интерфейсе Drive, затем `drive.mount()` и `cp`. Не читать оригинал через ярлык: ошибка квоты проявляется как `Input/output error`.
- **rclone:** `rclone backend copyid gd: <ID> gd:wiut/ → rclone md5sum → rclone copy ... --multi-thread-streams 8`.
- **gdown 6.4.0:** `gdown <ID> --cookies-from-browser firefox --continue` (Chrome 127+ шифрует cookies так, что gdown их не читает).
- У бесплатного Drive 15 ГБ, поэтому каждый участник копирует не больше двух файлов. Храним в частных Kaggle-датасетах `wiut-orig` и `wiut-proxy`; на ноутбуке — только прокси.

Что известно о тесте (оценка)

- Таймкод камеры в режиме Rec-Run. Под пропущенными номерами записано: C3898–C3901 — 370 с, C3903–C3904 — 653 с. Значит, тест, вероятно, составлен из этих клипов, всего ≈17 мин ≈ 0.28 ч [org].
- Часть теста, вероятно, снята вечером у заката: приоритет — низкое солнце, автоэкспозиция, длинные тени. Ночь, ИК и дождь — в конец плана.
- Две сессии с разницей 5.5 ч — вероятно, две установки штатива. Сцену нужно регистрировать отдельно под каждое видео (раздел 4.5).
- По PDF в тесте есть события, которых нет в сэмплах, а сэмплы имеют «то же разрешение и частоту кадров», что и тест. При этом в PDF и примере GT фигурирует 25 fps. Время всегда считаем как `frame_index / CAP_PROP_FPS`, fps никогда не хардкодим.

3.2 Бюджет времени: декодирование решает всё

При 29.97 fps лимит 3× даёт **100 мс на один исходный кадр на Part A и Part B вместе**. Для C3896 (10 200 кадров) это 1 021 с. Замер под WSL2 Ubuntu (8 vCPU из i5-12450H, Python 3.10, `opencv-python-headless==4.13.0.92`, `av==17.1.0`), шум ±25%:

Путь (4K 4:2:2 10-bit, если не указано иное)	мс на кадр	Кратность длительности
cv2.read() харнесса, Linux-колесо OpenCV	28–33	0.84–1.0×
cv2.read(), Windows-колесо (FFmpeg 4.4, swscale в один поток)	45–60	1.35–1.8×
cv2.grab() без конвертации	21.8	0.65×
grab() на каждом кадре + retrieve() на каждом 2-м / 3-м	29.2 / 27.0	0.87 / 0.81×
PyAV, только декод, thread_type="AUTO"	20–21	0.60–0.64×
PyAV с потоками по умолчанию (SLICE) — ловушка	99	3.0×
PyAV + skip_loop_filter	18.5	0.55×
PyAV + reformat(1280×720, bgr24) на каждом 2-м кадре + skip_loop_filter	21.1	0.63×
1080p25 4:2:0: read() харнесса / PyAV декод	4.1 / 1.8	0.10 / 0.05×

Факты, которые определяют стратегию

- NVDEC на Turing (T4, GTX 1650) декодирует H.264 только до High 8-bit 4:2:0. 4:2:2 10-bit — только Blackwell. -hwaccel cuda пишет «Failed setup for format cuda» и молча уходит на CPU.
- Дороже всего конвертация YUV → BGR, а не декод. Не вызывать retrieve() на кадрах, которые не обрабатываем.
- В long-GOP H.264 stride не уменьшает объём декодирования: экономит только конвертацию и время модели.

Следствия

- Время харнесса зависит от сборки OpenCV: Linux-колесо 4.13.0.92 (FFmpeg 8.0.1, многопоточный swscale) в 1.4–2 раза быстрее Windows-колеса.
- OPENCV_FFMPEG_CAPTURE_OPTIONS харнесс не ускоряет, а с avdiscard ломает метки idx/fps. Не трогать.
- При 4K GPU загружен лишь на 15–25% даже с YOLO26m@1280 и stride 2: крупная модель почти бесплатна, дорого увеличивать вход и тайлить (это swscale на CPU).

Перенос на машину оценки (оценка). Ближайший облачный аналог спецификации — AWS g4dn.2xlarge, но там 8 vCPU — это 4 физических ядра Xeon P-8259L. По PassMark множитель к ноутбуку: 1.5× (оптимистично), 1.9× (центрально), 2.3× (пессимистично). Если «8 CPU cores» — это 8 физических ядер, всё будет заметно быстрее [орг.].

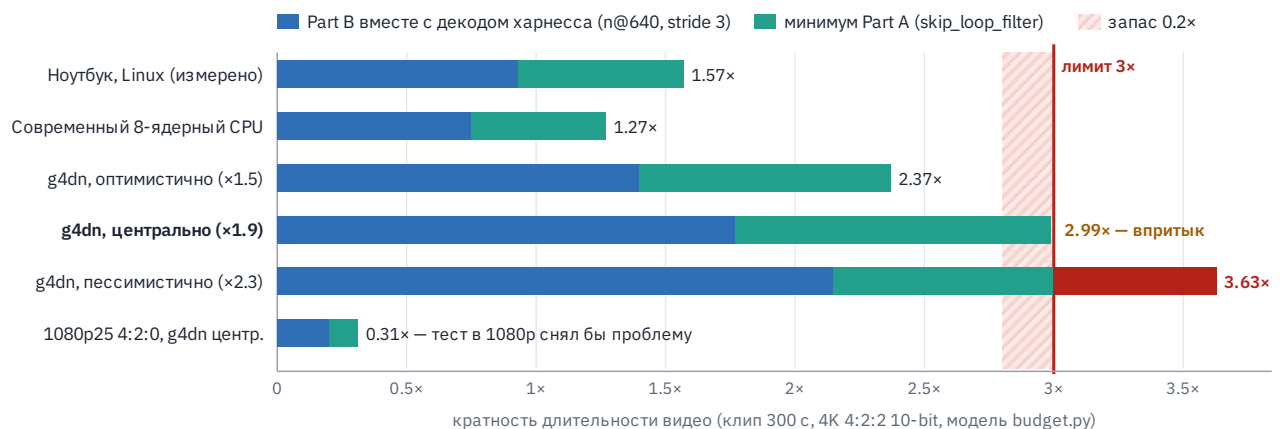


Рис. 1. Сколько остаётся на Part A. Синий — Part B вместе с обязательным декодом харнесса, зелёный — минимально возможный Part A. В центральном сценарии g4dn с Part B не укладывается ничего разумного; сохранить Part B можно примерно за 2.75× (Part A: m@640, stride 6, NONREF). Цифры — модель по замерам на ноутбуке, не замер на целевой машине.

3.3 Метрика Part A: что из неё следует

Размывание: когда вообще предсказывать класс

Обозначим: p — вероятность, что класс есть в тесте; f — ожидаемый F1 на нём; q — вероятность хотя бы одного ложного срабатывания на всём тесте, если класса нет; $\bar{S}$ — средний F1 остальных N классов. Если класс есть, он попадает в S при любом нашем решении, и предсказание даёт $+f/(N+1)$. Если класса нет, одно ложное срабатывание добавляет в S класс с нулём и отнимает $\bar{S}/(N+1)$ — ровно столько же, сколько сто ложных. Проверено на официальном скорере: $0.5556 \rightarrow 0.4167$ при одном ложном `fire_smoke` и столько же при трёх.

Предсказывать класс, если $p \cdot f > (1-p) \cdot q \cdot \bar{S}$. Порог: $p^* = q \cdot \bar{S} / (f + q \cdot \bar{S})$. Перевод FP/ч в q : $q = 1 - \exp(-\lambda \cdot T)$, $T \approx 0.28$ ч

$f \setminus q \cdot \tilde{S}$	0.05	0.10	0.20	0.30	0.40
0.1	0.33	0.50	0.67	0.75	0.80
0.2	0.20	0.33	0.50	0.60	0.67
0.3	0.14	0.25	0.40	0.50	0.57
0.5	0.09	0.17	0.29	0.37	0.44

λ , ложных в час	0.1	0.5	1	2	5
q при $T \approx 0.28$ ч	0.03	0.13	0.25	0.43	0.75

- **Аудит на сэмплах ограничен:** ноль ложных за 18.4 мин доказывает лишь $\lambda < 10/\text{ч}$ (95%, «правило трёх»). Поэтому `fire_smoke` и путь «статичный блок» в `road_obstacle` по умолчанию выключены.
- **Пороги — по классам:** чем меньше уверенность, что класс встретится, тем выше порог. На игрушечном детекторе оптимальный порог рос так: $p = 0.9 \rightarrow 0.55$, $p = 0.5 \rightarrow 0.60$, $p = 0.3 \rightarrow 0.65$, $p = 0.1 \rightarrow 0.75$.
- **Правило для кандидата** (Липтон–Элкан–Нараянасвами): выдавать, если $\sum_{\tau} P(\text{совпадёт при } \tau)/3 > F1_c/2$. Кандидат, который совпадёт только при $\tau = 0.3$, повышает $F1$, пока $F1_c < 0.67$. Лучше сначала исправить границы.

Допуск по границам

Тип ошибки (L — длина GT)	$\tau = 0.3$	$\tau = 0.5$	$\tau = 0.7$
сдвиг: $L(1-\tau)/(1+\tau)$	0.538L	0.333L	0.176L
расширение с каждой стороны: $L(1-\tau)/(2\tau)$	1.167L	0.5L	0.214L
сужение с каждой стороны: $L(1-\tau)/2$	0.35L	0.25L	0.15L
один конец удлинён: $L(1-\tau)/\tau$	2.33L	1.0L	0.43L
один конец укорочен: $L(1-\tau)$	0.7L	0.5L	0.3L

Длина L	сдвиг $\tau=0.3$	$\tau=0.5$	$\tau=0.7$
2 с	1.08 с	0.67 с	0.35 с
3 с	1.62 с	1.00 с	0.53 с
5 с	2.69 с	1.67 с	0.88 с
10 с	5.4 с	3.3 с	1.8 с
30 с	16 с	10 с	5.3 с
120 с	65 с	40 с	21 с

- **Короткие события** (`accident`, `near_miss`, `red_light`, `failure_to_yield`, `solid_line_crossing`, 1–5 с): ошибка каждой границы $\sigma \leq 0.3\text{--}0.5$ с. Границы берём от геометрии, а не от плато вероятности классификатора.
- **Длинные события** (`congestion`, `stopped_vehicle`, `fire_smoke`, `road_obstacle`) терпят секунды ошибки; их враги — фрагментация и позднее начало из-за окна подтверждения. Пример: если выдавать «стоит» в момент, когда выполнен 10-секундный критерий, $IoU = (L-10)/L$; для $L = 15$ с это 0.33.
- Слишком длинный сегмент стоит дешевле слишком короткого: при $\sigma \approx 0.5\text{--}1$ с расширяем каждую сторону на $\sigma/2$.

Фрагментация, склейка, дубликаты (проверено официальным сопоставителем)

Случай (GT = [0, 10], если не указано иное)	TP/FP/FN $\tau=0.3$	$\tau=0.5$	$\tau=0.7$	средний F1
идеально	1/0/0	1/0/0	1/0/0	1.000
разрез [0,5] + [5.5,10]	1/1/0	1/1/0	0/2/1	0.444
симметричный разрез [0,4.75] + [5.25,10]	1/1/0	0/2/1	0/2/1	0.222
разрез на 3 части	1/2/0	0/3/1	0/3/1	0.167
два GT [0,5] и [7,12] склеены в [0,12]	1/0/1	0/1/2	0/1/2	0.222
лишний блип 0.5 с рядом с идеальным совпадением	1/1/0	1/1/0	1/1/0	0.667
авария 3 с выдана окном 60 с	0/1/1	0/1/1	0/1/1	0.000
выданы [0,3] и [2,10] → харнеис оставит только [0,3]	1/0/0	0/1/1	0/1/1	0.333

Игрушечная симуляция склейки разрывов g и минимальной длительности m : короткие события — $F1\ 0.28 \rightarrow 0.93$ при $g = 1\text{--}2$ с, $m = 1\text{--}1.5$ с; длинные — $0.03 \rightarrow 0.93$ при $g = 6\text{--}10$ с, $m = 10\text{--}15$ с. Эмпирическое правило: **$g \approx 3\text{--}5 \times$ типичного выпадения, но меньше интервала между событиями; $m \approx \frac{1}{3}\text{--}\frac{1}{2}$ самого короткого правдоподобного события.** Больше ~10 событий одного класса на 5 минут почти наверняка означают сломанное правило; FP суммируются по всем видео, поэтому одно плохое видео портит класс для всех.

Малые выборки. Monte Carlo (recall 0.7, $IoU \sim \text{Beta}(6,2)$, ≈ 1 FP на тест): при $n = 1$ GT средний $F1\ 0.45$, $sd\ 0.36$, $P(F1 = 0) = 0.30$; при $n = 3$ — $0.59/0.21/0.03$; при $n = 10$ — $0.67/0.11/0$. Разницу меньше ~0.1 на dev считать шумом, предпочитать изменения, обоснованные механизмом.

3.4 Метрика Part B: механика и ловушки

Разметка кадров (порядок проверок в `frame_label`): кадр внутри аварии $[s, e] \rightarrow$ игнор; иначе кадр в $[s-5, s) \rightarrow$ **позитив** (сильнее игнора `near-miss`); иначе кадр в $[s_{nm}-5, e_{nm}]$ вокруг `near_miss` $\rightarrow$ игнор; всё остальное — негатив. AP

— пошаговая (как в sklearn) по кадрам всех видео, нормированная: $AP = \max(0, (AP_{\text{raw}} - r)/(1 - r))$. Тревога — максимальная серия кадров со скором ≥ 0.5 ; серии с разрывом < 2 с сливаются **до** отбрасывания тревог, начавшихся на игнорируемых кадрах. Каждой аварии сопоставляется самая ранняя свободная тревога с началом в $[s-10, s)$; остальные ложные. $F1_{\text{alarm}} = 2m/(A + N)$; $TTA = s - \text{начало тревоги}$.

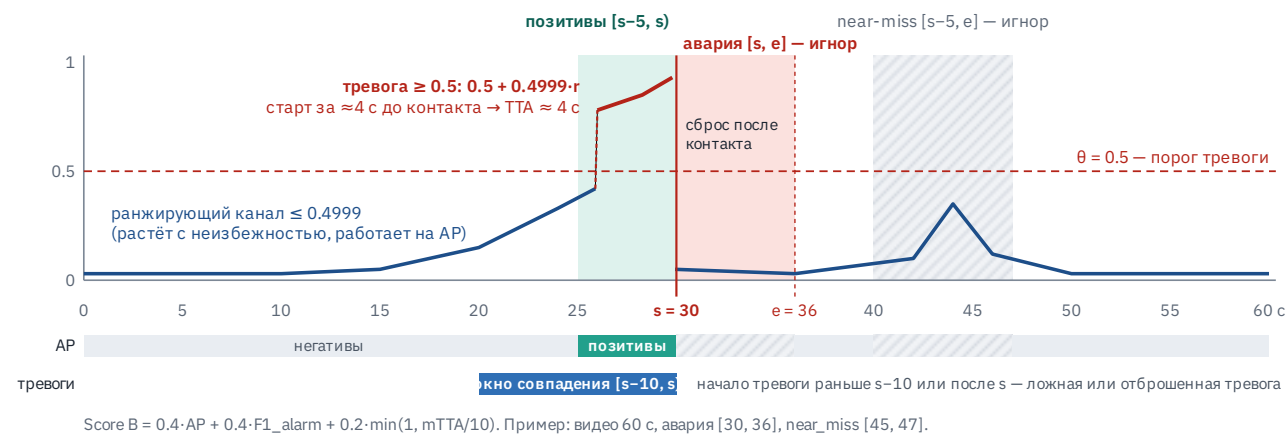


Рис. 2. Целевая форма сигнала Part B. Монотонный рост $0.5 \rightarrow 0.9$ на $[20, 30]$ даёт $\text{Score B} = 1.000$; тревога, стартовавшая в момент контакта, — 0.000 .

Кривая риска (видео 60 с, авария [30, 36], near_miss [45, 47])	AP	F1	mTTA	Score B
0.9 только на [25, 30]	1.00	1	5.0	0.900
монотонный рост $0.5 \rightarrow 0.9$ на [20, 30]	1.00	1	10.0	1.000
плоско 0.9 на [20, 30]	0.44	1	10.0	0.776
поздно: 0.9 на [29, 30]	0.20	1	1.0	0.500
тревога стартует в момент контакта s	0	0	0	0.000
держится 6 с после e	0.22	1	3.0	0.547
ловушка слияния: блип [18, 18.5) + истинная [20.3, 30), зазор 1.8 с	0.43	0	0	0.172
то же, зазор 2.8 с	0.43	0.67	9.7	0.632
0.49 на [25, 30] — работает только AP	1.00	0	0	0.400
0.49996 на [25, 30] — округляется до 0.5	1.00	1	5.0	0.900
устойчивая тревога 0.9 на $[s-15, s)$	0.25	0	0	0.102

Ловушки

- Опоздание смертельно:** тревога, начавшаяся в момент контакта или позже, попадает на игнор и выбрасывается. Цель — старт за $\geq 0.5-1$ с до контакта.
- Слияние с ранней ложной тревогой** (< 2 с) переносит начало раньше; если раньше $s-10$ — совпадения нет.
- Слияние с серией, начавшейся на игноре** (внутри аварии или окна near-miss): слитая тревога получает игнорируемое начало и выбрасывается целиком.
- Длинные тревоги не работают; вторая тревога перед той же аварией — ложная; высокие скоры после e — негативы для AP.

Оптимальная форма сигнала

- Два канала: ранжирующий ≤ 0.4999 всегда, тревожный ≥ 0.5 только при уверенном неизбежном конфликте.
- Монотонность по неизбежности (падению TTC); физические шкалы, без нормировки по видео.
- Лёгкое сглаживание (~ 0.2 с), без задержек подтверждения.
- Сброс после контакта, подавление пары ≤ 10 с; цепные аварии не глушить.
- Возраст тревоги ≤ 8 с без принудительного повторного включения.
- Порог 0.5 — по бюджету ложных тревог: ≤ 0.5 на тест $\approx \leq 1.8/\text{ч}$ на сэмплах вне окон near-miss. mTTA не гнаться: $0.3 \rightarrow 2$ с даёт лишь $\approx +0.03$.

$F1_{\text{alarm}}$ при малом числе аварий (recall 0.6): 1 авария — 0.75 без ложных, 0.46 при одной ложной, 0.33 при двух; 3 аварии — 0.75 / 0.62 / 0.53; 10 аварий — 0.75 / 0.71 / 0.67. Каждая ложная тревога при 1–3 авариях стоит 0.05–0.3 F1. При малом числе аварий разброс Score B ≈ 0.1 , поэтому Part B ограничиваем $\approx$ одной человеко-неделей.

3.5 Открытые развилки

Вопрос	Варианты	Как решаем
Пин OpenCV	Отчёт: <code>opencv-python-headless==4.13.0.92</code> (подкреплено замером харнесса под Linux). Каталог: <code>opencv-python==4.14.0.94</code> (его тянут <code>ultralytics</code> и <code>trackers</code> ; 4.14 не бенчмаркался). Без пина встанет 5.0.0.93 с миграцией API.	Замером полного <code>run_submission.py</code> на реальных сэмплах. Два пакета OpenCV в одном окружении не ставить (для Ultralytics — пакет <code>ultralytics-opencv-headless</code>).
Лицензии	Ultralytics считает обученные веса AGPL-3.0; AGPL не допускает доп. ограничений вроде «только некоммерческое». Данные UA-DETRAC, MIO-TCD, ACCIDENT — NC-SA.	До обучения детектора выбрать вариант (a) или (b) из раздела 7.6; при сомнении — вопрос организаторам.
Исключение в Part B	Код харнесса сохраняет события Part A, PDF называет это пустым предсказанием.	Уровень ТЗ («жертвуем Part B») не планируем до ответа [орг.].
Конвенции <code>congestion</code> , <code>stopped_vehicle</code>	H1 (очередь на красный не считается) против H2 (любая остановка всех полос ≥ 20 с); старт <code>stopped_vehicle</code> — остановка или +10 с.	Размечаем dev в обоих вариантах, по умолчанию H1 и «старт = остановка» [орг.].

4. Архитектура решения

Part A работает офлайн: может смотреть в будущее, делать несколько проходов и обращаться к файлу в любом порядке. Part B строго причинный и **не использует никаких выходов Part A**; общими могут быть только веса и код. Подход гибридный: обучаемые модели дают восприятие (детекция объектов, огонь и дым, подтверждение аварий), а решения о событиях принимают правила на траекториях в метрах и на геометрии сцены. Так границы событий привязаны к физическим моментам, а поведение объяснимо и настраивается на малом dev-наборе.

4.1 Общая схема Part A

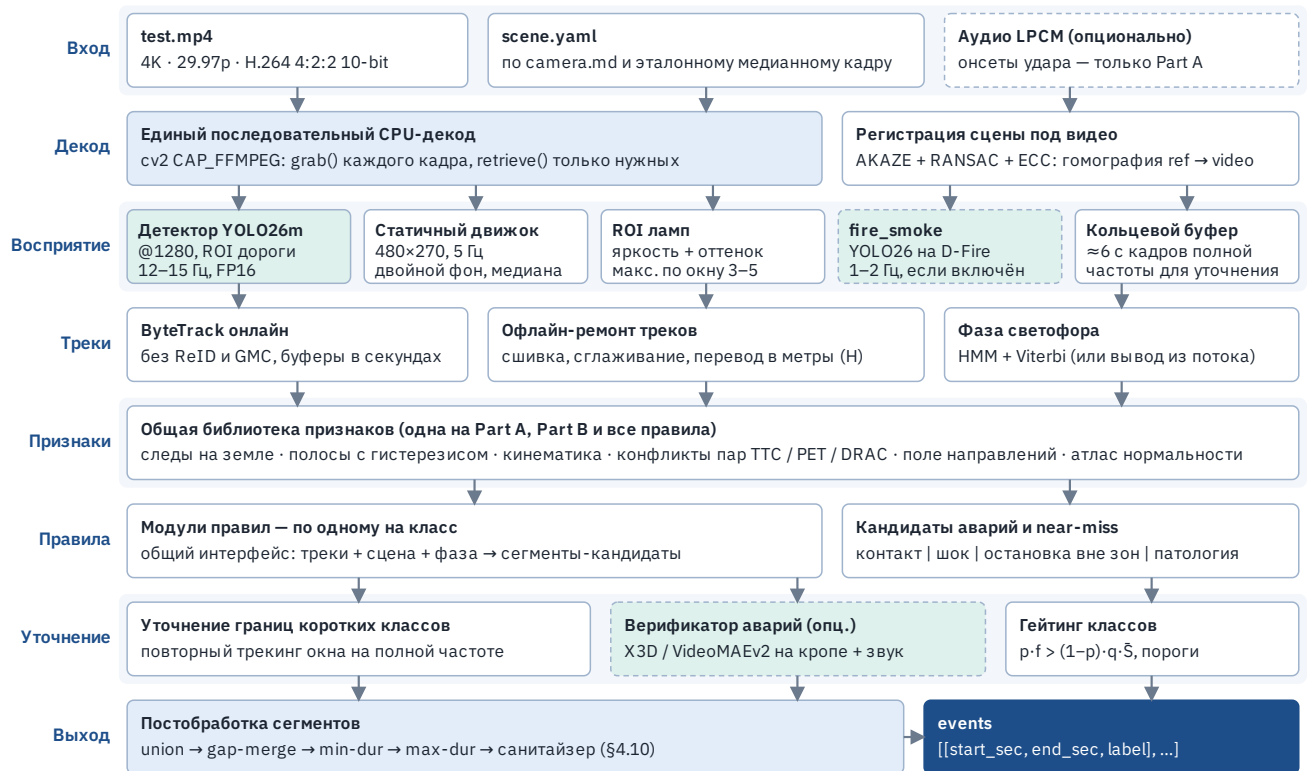


Рис. 3. Конвейер Part A. Синие блоки — узкие места по времени (декод) и выход; зелёные — обучаемые модели; пунктир — опциональные блоки, которые включаются только по результатам dev и бюджету.

Ключевые принципы

- Одно декодирование на Part A.** Все потребители кадров берут их из одного последовательного прохода. При 4K любое повторное чтение окон стоит 0.4–0.5× длительности и больше, поэтому окна уточнения собираем из кольцевого буфера во время основного прохода. Произвольный доступ PyAV — только для 1080p или при явном запасе.
- Кэш для разработки:** детекции и треки каждого сэмпла сохраняются в .npz с ключом «хэш видео + хэш конфига». Перезапуск правил занимает секунды, и пороги подбираются против dev-разметки через evaluate.py. В сабмишене кэш не используется.
- Part A не создаёт RiskEstimator внутри себя:** это повторно оплатило бы декод (0.5–1.3× при 4K). Те же TTC-признаки Part A считает по своим трекам почти бесплатно (≈0.005×). FAQ разрешает Part A использовать кривую риска, обратное — нет.
- Хуки для сайта сразу:** detect_events(path, fast=False, on_progress=None); треки, зоны и track_ids сохраняются для каждого события, чтобы рисовать их в плеере.

4.2 Декодирование и бюджет-планировщик

Жёсткая остановка Part A: $T_A = 3 \cdot \text{dur} - 1.2 \cdot (\text{Part B, включая декод харнесса}) - 0.2 \cdot \text{dur} \approx 1.7 \times$ на ноутбуке (Linux), $\approx 1.1 \times$ на g4dn оптимистично, $\approx 0.7 \times$ на g4dn центрально (меньше минимума Part A 0.91–1.22×)

1. **План строится только из meta** (fps, width, height, n_frames) и таблицы стоимостей, измеренной на T4 и похожем CPU. Из плана выводятся stride, размер входа, число окон уточнения и включённые опции. Такой план детерминирован.
2. **Этапы Part A упорядочены по ценности:** детекция и трекинг → правила → уточнение границ → тайлинг → верификаторы. При отсечке первыми выпадают уточнение, тайлы и VLM.
3. **Аварийная проверка:** в начале detect_events читаем 60 кадров так же, как харнесс, и получаем h — стоимость харнесса в долях длительности (2–4 с). Если h больше табличного более чем на 30% — берём уровень легче и логируем. Уровень можно зафиксировать переменной WIUT_TIER.
4. **Защита Part B:** каждые 32 вызова step() прогнозирует now + оставшиеся_кадры × EMA(dt); если прогноз выходит за «дедлайн – 5 с», stride удваивается. Таймаут обнуляет обе части, поэтому это важнее качества.
5. **Всё тяжёлое — при импорте:** загрузка моделей, контекст CUDA, cuDNN, прогревы. В step() не копировать 4K-кадр (9 мс): сразу cv2.resize в заранее выделенный буфер (1–2 мс).
6. **Part B с фиксированным лагом L = 3:** рабочий поток считает детектор, трекер и TTC для кадра t, вызов step(t + L · stride) ждёт результата. Решение остаётся причинным и детерминированным, GPU перекрывается с декодом харнесса; цена — 0.1–0.3 с ТТА.

Уровень	Условие по h	Part A	Part B
T0 богатый	$h \leq 0.8$	skip_loop_filter, YOLO26m@1280 на ROI, stride 2 (15 Гц), до 40 окон по 8 с на полной частоте, статичный движок 5 Гц, огонь 1 Гц	s@960, stride 2, с перекрытием
T1 стандарт	$0.8 < h \leq 1.2$	s@1280 или m@960, stride 3 (10 Гц), ≈20 окон, статичный движок	n@640, stride 3
T2 экономный	$1.2 < h \leq 1.55$	NONREF (если в GOP есть неопорные B-кадры) + skip_loop_filter, s@960/1280, без повторного чтения	n@640, stride 3–4
T3 жертвуем Part B	$h > 1.55$	до 2.4× (m@1280, stride 2)	reset() бросает исключение → risk = []. Не планируем до ответа [огр.]

Прочие статьи (оценка): регистрация сцены 1–1.5 с на видео; аудио ≤ 2 с на минуту; статичный движок ≈ 5 мс на вызов; фазовая корреляция ≈ 6 мс; VLM 20 запросов × 3–8 с = 60–160 с, то есть 1.0–2.7× для минутного клипа — поэтому VLM по умолчанию выключен.

Бэкэнд: декод только на CPU через OpenCV cv2.VideoCapture(path, cv2.CAP_FFMPEG) — тот же бэкэнд, что у харнесса. PyAV 17.1.0 (thread_type="AUTO", frame.reformat(w, h, "bgr24")) — только если бенчмарк покажет явный выигрыш и пройдёт проверку совпадения пикселей и числа кадров с cv2. Инференс — PyTorch FP16, cudnn.benchmark=False, фиксированный размер батча. TensorRT и torch.compile не в пути по умолчанию (движок привязан к устройству, компиляция попадает в бюджет). ONNX Runtime не использовать: версии ≥ 1.24 без колёс ср310, с 1.27 GPU-сборки под CUDA 13. decord заброшен, NVDEC-пути (PyNvVideoCodec, TorchCodec CUDA) исключены.

4.3 Детектор

Семейство	COCO mAP50-95	T4 TRT FP16, мс	Лицензия	Роль
YOLO26 n / s / m / l / x (январь 2026, ultralytics==8.4.163)	NMS-free: 40.1 / 47.8 / 52.5 / 54.4 / 56.9; с NMS: 40.9 / 48.6 / 53.1 / 55.0 / 57.5	1.7 / 2.5 / 4.7 / 6.2 / 11.8	AGPL-3.0	основной m — Part A, s/n — Part B
YOLO11 n–x	39.5 / 47.0 / 51.5 / 53.4 / 54.7	1.5 / 2.5 / 4.7 / 6.2 / 11.3	AGPL-3.0	запасной тот же код
RF-DETR N / S / M / L (rfdetr==1.11.0)	48.4 / 53.0 / 54.7 / 56.5 (вход 384–704)	2.3 / 3.5 / 4.4 / 6.8	Apache-2.0 (XL/2XL — PML 1.0, не брать)	запасной полностью пермиссивный стек
D-FINE N–X / RT-DETRv4 S–X	42.8–55.8 / 49.8–57.0	2.1–12.9 / 3.7–12.9	Apache-2.0	альтернатива
DEIMv2	43.0–57.8	2.3–13.8	некоммерческая	избегать

- **Задержки TRT и PyTorch несравнимы:** PyTorch FP16 с пред- и постобработкой медленнее в 1.2–2 раза; YOLO26m@1280 ≈ 13–22 мс на кадр (оценка). Локальная GTX 1650 — sm_75, как T4, но без тензорных ядер; скорость меряем только на T4 (Kaggle, Colab).

- **Вход при 4K:** кроп по ограничивающему прямоугольнику проезжей части из camera.md (с запасом вокруг бордюров и переходов), затем 1280 по длинной стороне. Машина шириной 60 px в 4K после масштабирования ≈ 20 px — достаточно. Критерий: 5-й перцентиль роста дальнего пешехода ≥ 12 px [camera.md]. Тайлинг — только один тайл дальней зоны на 5 Гц, если дальние пешеходы меньше ≈ 16 px.
- **Суперклассы, трекинг без учёта класса:** `vehicle` (car, bus, truck), `two_wheeler` (motorcycle, bicycle), `person`, `animal`; класс трека — голосованием. До трекинга — class-agnostic NMS при IoU 0.7 (NMS-free голова иногда выдаёт на один объект и car, и truck).
- **Ловушка для jaywalking:** COCO помечает райдера как `person`. Человека выбрасываем, если его бокс перекрыт двухколёсником и ступни внутри бокса двухколёсника, либо если медианная скорость > 3.5 м/с дольше 2 с.

Дообучение детектора — стоит того (тест снят той же камерой)

1. Учитель: YOLO26x на 1920 с ТТА на кадрах сэмплов.
2. Фильтр временной согласованности: бокс должен иметь пару с IoU > 0.5 в соседнем кадре.
3. ≈ 300 – 600 ключевых кадров исправить вручную (раздел 6.7), одно видео целиком отложить (hold-out).
4. Дообучить s и m на суперклассах и светофоре с `seed=0`, `deterministic=True`; добавить 10% grayscale, HSV-jitter и 20–30% кадров неподвижных дорожных камер (с учётом лицензионной развилки, раздел 7.6).
5. Смесь: $\approx 40\%$ наших кадров (с оверсэмплингом) + $\approx 40\%$ дорожных камер + $\approx 20\%$ COCO (люди, двухколёсные, животные, светофоры) как rehearsal.
6. Оценивать по **событийному F1 на dev**, а не только по mAP.

4.4 Трекер и офлайн-ремонт треков

Трекер	Библиотеки (лицензия)	Вердикт
ByteTrack	Ultralytics (AGPL), Roboflow trackers 2.6.1 (Apache-2.0), FoundationVision (MIT)	основной вторая ассоциация по детекциям с низкой уверенностью вытаскивает машины, перекрытые в очереди
OC-SORT	trackers, BoxMOT (AGPL)	запасной лучше на нелинейном движении
BoT-SORT / TrackTrack без GMC и ReID	Ultralytics	допустимо (<code>gmc_method: none</code> , <code>with_reid: False</code>)
Deep OC-SORT, StrongSORT	BoxMOT	не нужны: требуют ReID, камера неподвижна

Стартовые параметры (настраиваются на dev): порог детектора 0.10; `track_high_thresh` 0.40, `track_low_thresh` 0.10, `new_track_thresh` 0.50, `match_thresh` 0.8; `gmc_method: none`, `with_reid: False`; правила используют только треки длиной ≥ 0.5 – 1 с со средней уверенностью ≥ 0.4 . Трекер создаётся заново на каждое видео (`persist=True` между видео не использовать).

Ловушки трекеров

- **`fuse_score`.** В Ultralytics по умолчанию `fuse_score=True`: условие первого этапа — $\text{IoU} \times \text{score} \geq 0.2$, новорождённые треки проходят только при $\text{IoU} \times \text{score} \geq 0.3$ и предсказываются с нулевой скоростью. Машина на 60 км/ч при stride 3 даёт сырой $\text{IoU} \approx 0.38$; при score 0.6 это $0.23 < 0.3$ — трек не подтверждается. Stride 3 при 29.97 fps допустим только с `fuse_score=False`, либо с `trackers` (там IoU не умножается на score), либо с пониженным порогом рождения.
- **Буферы считаются по-разному:** Ultralytics — `max_frames_lost = track_buffer` в обработанных обновлениях без масштабирования по fps; FoundationVision и `trackers` — `frame_rate/30` · `track_buffer`. Решение — один модуль `tracker_config.py`, буферы задаются в секундах. Пример: 3 с при 29.97/stride 2 (15 Гц) — это 45 обновлений в Ultralytics и `track_buffer = 90` при `frame_rate = 15` в `trackers`.

Офлайн-ремонт треков (только Part A)

- **Движущиеся объекты:** зазор ≤ 2 – 4 с; $|p_A + v_A \cdot \Delta t - p_B| \leq 2 \text{ м} + 0.5 \cdot |v| \cdot \Delta t$; тот же суперкласс; отношение площадей в $[0.5, 2]$; опционально сходство HSV-гистограмм; решение — венгерский алгоритм.
- **Стоящие объекты:** зазор ≤ 30 с при совпадении положения в пределах ≈ 1.5 м, плюс «ghost-боксы»: потерянный стоящий трек живёт, пока $\geq 40\%$ его бокса статично или $\geq 60\%$ остаётся передним планом долгого фона.

- **Разрез по неправдоподобным скачкам** (ускорение $> 8 \text{ м/с}^2$, поворот курса $> 90^\circ$ за 0.2 с) вне окон кандидатов в аварию: разрезать безопаснее, чем склеивать.
- **Сглаживание**: фильтр Хампеля $\rightarrow$ интерполяция разрывов $\leq 1 \text{ с}$ $\rightarrow$ Savitzky–Golay (порядок 2) или RTS.
- Переключения ID порождают ложные `wrong_way`, `illegal_u_turn`, `solid_line_crossing`, поэтому в «подозрительном окне» правила с разворотом курса и скачками Δv не срабатывают.

4.5 Геометрия сцены и регистрация

Хардкодить факты о сцене из camera.md (направления полос, стоп-линии) разрешено и ожидается. Все координаты храним **нормированными (0–1)**, чтобы одинаково работать с 4K, 1080p-прокси и загрузками в демо. Слои рисуем на **медианном кадре** (медиана по кадру раз в 2 с), где видна чистая разметка без машин.

```
reference: {video: C3896, plate: weights/scene/ref_plate_960.png, mask: weights/scene/ref_mask_960.png}
homography_pts: {img: [[u,v], ...], world_m: [[x,y], ...]} # >= 8 точек на плоскости дороги
carriageway: [poly, ...]
non_carriageway: {sidewalk: [...], island: [...], parking_bay: [...], bus_stop: [...]}
lanes: [{id: N1, poly: [...], approach: N, dir: centreline_polyline, allowed: [through, left]}]
separators: [{id: s1, polyline: [...], type: solid|dashed|double_solid|solid_dashed, crossable_from:
left|right|none}]
stop_lines: [{approach: N, polyline: [...], signal_group: G1}]
crossings: [{id: X1, poly: [...], signalized: true, curb_zones: [poly_a, poly_b]}]
intersection_box: poly
gates: {entry: {...}, exit: {...}} # для OD-матрицы
movements_prohibited: [{from: N, move: left}] # знаки 3.18.x [camera.md]
u_turn: {prohibited: [poly], allowed: [poly]} # знак 3.19, двойная сплошная
right_on_red: {N: false} # true только при знаке 5.42
signals: [{group: G1, lamps: {red: roi, yellow: roi, green: roi}, housing: roi}]
static_ignore: [poly] # оверлей времени, деревья, небо
```

Гомография плоскости земли

- ≥ 8 точек на дороге, кликнутых вручную: углы разметки, концы стоп-линий, углы полос перехода. Метрика — из измерений (ширина полосы, иначе 3.5 м; шаг перехода). Длину штрихов разметки как эталон не использовать.
- `cv2.findHomography(img, world, method=0)` — наименьшие квадраты. **Не RANSAC с порогом 3.0:** порог в единицах назначения — это 3 м, он ничего не отсеивает.
- Проверки: невязка каждой точки $\leq 0.1\text{--}0.2 \text{ м}$; разделители после проекции параллельны; ширины полос $\pm 10\%$; длины машин $\approx 4.5 \text{ м}$; скорость свободного потока правдоподобна.
- Надёжная кинематика — только где `rptm` (пикселей на метр вдоль движения, по минимальному сингулярному значению якобиана H^{-1}) ≥ 8 . При `rptm` 8 и окне SG 1.5 с шум ускорения $\approx 0.3\text{--}0.5 \text{ м/с}^2$; при `rptm` 4 — $\approx 2\text{--}2.7 \text{ м/с}^2$ (оценка).

Регистрация сцены под каждое видео (обязательна)

1. Медианный кадр по 48 кадрам, взятым раз в $\approx 6.7 \text{ с}$ из основного прохода ($\approx 0.35 \text{ с}$).
 2. CLANE (clip 2.0, тайлы 8×8) на эталонном и текущем кадре.
 3. AKAZE (порог 5e-4) внутри маски стабильных структур; BF-Hamming, тест отношения 0.8; `findHomography` RANSAC 2.0 px в масштабе 960×540 .
 4. Уточнение ECC `MOTION_HOMOGRAPHY` : 480×270 , затем 960×540 .
 5. Жёсткие проверки: `inliers` ≥ 40 , доля ≥ 0.25 , `RMSE` $\leq 1.5 \text{ px}$, `ECC` ≥ 0.60 , признаки и ECC расходятся $\leq 12 \text{ px}$ в 4K. Смещение $< 4 \text{ px}$ $\rightarrow$ тождество.
 6. Детектор скачка по фазовой корреляции на 0.2 Гц; при скачке — кусочная `H(t)`.
- Синтетика: сдвиги до 134 px и зум 3% (сумерки, тени, 60% статичных пробок) восстанавливаются с ошибкой $\leq 1.3 \text{ px}$; чужая сцена отвергается. Перенос: полигоны — `perspectiveTransform` с $H_{4k} = S \cdot H \cdot S^{-1}$, $S = \text{diag}(4, 4, 1)$; маски — `warpPerspective` (nearest); гомография земли $G_{\text{cur}} = G_{\text{ref}} \cdot H_{4k}^{-1}$; ROI ламп — плюс локальный NCC-поиск $\pm 40 \text{ px}$.

Если регистрация не прошла: элемент или класс	Допуск, px 4K (оценка)	Действие
ROI ламп (<code>red_light</code> , <code>stop_line</code>)	5–8	Отключить классы
Сплошная (<code>solid_line_crossing</code>)	8–10	Отключить
Полигон перехода (<code>failure_to_yield</code>)	20–30	Отключить
Полосы (<code>illegal_turn</code> , <code>illegal_u_turn</code>)	30–50	Отключить
<code>wrong_way</code>	нужны направления, а не линии	Оставить на самокалиброванном поле направлений

Если регистрация не прошла: элемент или класс	Допуск, px 4K (оценка)	Действие
Проезжая часть (<code>stopped_vehicle</code> , <code>congestion</code> , <code>jaywalking</code> , <code>road_obstacle</code>)	≈ 50	Оставить; маска дороги — из тепловой карты движения видео
<code>accident</code> , <code>near_miss</code> , <code>fire_smoke</code>	не зависят	Оставить

Part B: отдельный `CausalRegistrar` — 4 кадра с шагом 0.4 с, готовность через 1.2 с, ≈ 1 px ошибки на синтетике, проверка дрейфа раз в 30 с; до готовности геометрические признаки риска получают меньший вес. Гомографию из Part A в Part B не передавать: она вычислена по будущим кадрам.

4.6 Общие библиотеки: следы, полосы, кинематика, конфликты

След на земле

- Нижняя середина бокса — ближайшая к камере точка следа.
- Ориентированный прямоугольник по классу (оценка): car 4.5×1.8 м, van 5.5×2.0, truck 8–9×2.5, bus 12×2.5, motorcycle 2.0×0.8, bicycle 1.8×0.6, пешеход — диск 0.5 м.
- Из него — **передняя и задняя точки** для `red_light`, `stop_line`, `failure_to_yield`: ошибка положения передка в 4.5 м при 15 м/с — это 0.3 с, весь допуск IoU 0.7 для 2-секундного события.
- Поправка на высокие машины: для полосы вычитаем медианное боковое смещение точек контакта (для `solid_line_crossing` обязательно).
- Флаги надёжности: бокс у края кадра (≤ 3 px) — обрезан; бокс перекрыт ближним — вес ниже.

Полосы с гистерезисом

Новая полоса фиксируется, если держится ≥ 0.5 с и центр отошёл от разделителя на ≥ 0.5 м. Пересечение разделителя — смена знака расстояния с $|d| \geq 0.3$ м по обе стороны не менее 0.4 с.

Кинематика (одна библиотека на всех)

- Part A: Хампель → RTS или центрированный SG. Part B: Калман с постоянной скоростью.
- Курс — только при скорости > 1 м/с, иначе держим последний.
- Состояния: стоит — < 0.5 м/с и смещение < 1 м за 2 с; ползёт — < 2 м/с; едет — > 3 м/с.
- Резкое торможение:** Δv за 1 с ≤ -3 м/с при $v_{до} \geq 4$ м/с. Момент начала офлайн — hinge-fit (двухсегментная кусочно-линейная $v(t)$). Мгновенное ускорение не пороговать: после SG(1.5 с) реальное торможение 0.5 с не доходит до -3.4 м/с²; onset по SG смещён раньше на ≈ -0.25 с — вычитаем как известную поправку.

Модуль конфликтов (`accident`, `near_miss`, Part B)

- ТТС:** постоянная скорость; первый момент $\tau \in [0, 5]$ с (шаг 0.1 с), когда капсулы следов (+0.3 м) пересекаются.
- Согласованность ТТС:** доля последних 0.8 с, где ТТС падал $\approx$ на 1 с/с — главный фильтр против следования в потоке.
- PET, T2, DRAC = $\Delta v^2 / (2 \cdot \text{зазор})$, скорость сближения, $d_{\min}$; MTTC — только попутные пары.
- Стартовые пороги: ТТС ≤ 1.5 с (SSAM) или PET ≤ 1.5 с; DRAC 3.35 м/с² (Archer 2005).
- Опоры, отбойники, островки — неподвижные капсулы (удар об объект).
- Готовый код: Two-Dimensional-TTC и SSMsOnPlane (MIT) — 2D TTC по ориентированным прямоугольникам, ≈ 0.036 с на 10^4 пар.

Поле направлений: сетка по плоскости земли (1 м) или изображению (64 px) — гистограмма курсов и доминирующее направление по всем трекам. Проверяет нарисованные вручную направления полос и служит запасным путём для `wrong_way`. **Атлас нормальности:** по ячейкам — направления, скорости и вероятность остановки; остановка там, где никто не останавливается, — сильный признак аварии или `stopped_vehicle`.

4.7 Светофор, огонь и дым, препятствия

Светофор [camera.md]

- ROI каждой лампы в `scene.yaml` (центр и радиус) переносится регистрацией, затем локальный NCC-поиск шаблона корпуса ± 40 px в 4K. Лампа ≈ 20 px в 4K — самый хрупкий элемент сцены.
- Признаки: 90-й перцентиль V в диске лампы, медианная S в кольце, циркулярная медиана оттенка по насыщенным пикселям.
- Калибровка по видео: Part A — 2-means по V за всё видео даёт порог «горит / не горит»; Part B — пороги с сэмплов плюс скользящая история.

4. **Положение важнее цвета:** горящая лампа — argmax яркости относительно её базового уровня, оттенок только подтверждает (переживает сдвиг баланса белого на закате).
5. Мерцание LED: берём максимум по окну из 3–5 отсчётов и сдвигаем переходы назад на $w/2$.
6. Фаза: HMM + Viterbi с минимальными длительностями (Part A), гистерезис (Part B). Красный+жёлтый трактуем как красный [орг.]; при блике — «неизвестно», держим предыдущее. Фиксированный цикл не предполагаем (в Ташкенте есть табло отсчёта и адаптивное управление).
7. Если сигнал не виден — фаза из поведения потока: очередь у стоп-линии и пересечения за 5 с; начало зелёного $\approx$ отъезд головы очереди минус 1–1.5 с. Правила, зависящие от красного, тогда работают только в режиме высокой точности. Запасной классификатор на кропах — Autoware `traffic_light_classifier` (MobileNetV2, ONNX, Apache-2.0).

Огонь и дым (`fire_smoke`)

- YOLO26n/s, дообученный на D-Fire (21 527 изображений, CCO) + 10–15% кадров-негативов сцены (стоп-сигналы, красные машины, блики, пыль, выхлоп); 1–2 Гц. Обучение $\approx$ 1–2 ч на Kaggle T4.
- Подтверждение: присутствие в 7 из 10 отсчётов за 5 с (уверенность: дым 0.35, огонь 0.45); на дороге или на машине, не на небе, $\leq 25\%$ кадра; **не привязан к машине, едущей быстрее 2 м/с дольше 3 с**; огонь — мерцание 3–12 Гц, вариация площади $CV \geq 0.1$, $R > G > B$; дым — нежёсткий поток вверх.
- Старт — перенос назад по трассе детектора с порогом 0.15. По умолчанию выключен.

Препятствия (`road_obstacle`)

- (а) Животные COCO (классы 15–19, уверенность ≥ 0.4 , ступни на проезжей части ≥ 1 с) — за тем же аудитом ложных.
- (б) Статичный блоб двойного фона: $0.05\text{--}6\text{ м}^2$, $\geq 70\%$ внутри проезжей части, $\geq 3\text{--}5$ с, не объясняется боксами ($\text{IoU} < 0.1$ для $\geq 80\%$ времени), текстурный тест против теней, верификатор YOLOE-26 с «запечённым» словарём или OWLv2 на кропе.
- Долгий фон замораживается до появления объекта, иначе объект «впитается». MOG2 поглощает остановку за $T \approx 0.105/(\alpha \cdot f)$ с, только если передавать `learningRate= $\alpha$` в каждый `apply()`. По умолчанию выключен.

4.8 Аварии и near-miss: кинематика + верификатор

Организаторы прямо пишут, что обученные модели полезнее всего для этих двух классов. Но сегмент аварии (от первого контакта до остановки всех участников) длится $\approx 2\text{--}6$ с (оценка), и для $\text{IoU} 0.7$ суммарный сдвиг границ должен быть $\leq 0.18 \cdot L$: ≈ 0.35 с для 2-секундного события. Такую точность даёт только траектория, поэтому **ядро — кинематика, а обучаемая модель только подтверждает**.

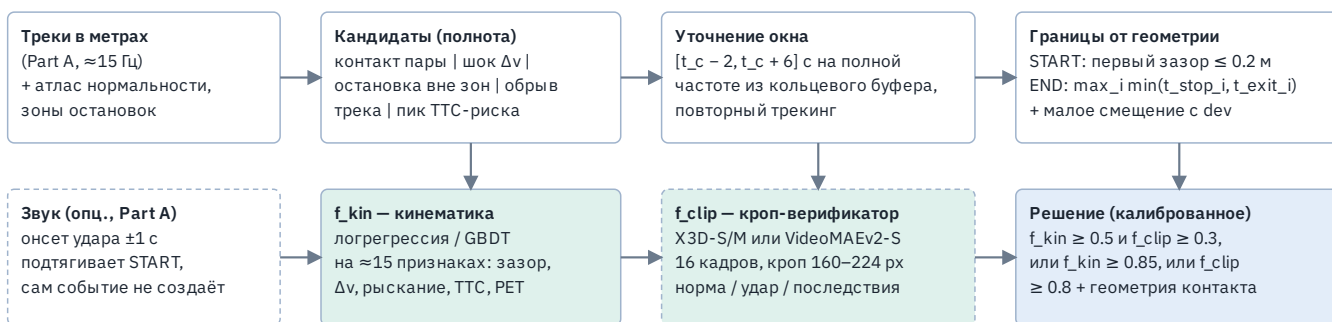


Рис. 4. Детектор аварий: кандидаты с упором на полноту, уточнение на полной частоте, границы только от геометрии, решение — по двум независимым семействам доказательств. Зелёные блоки обучаемые, пунктир — опционально.

(а) Кинематические признаки (ядро)

- 1. **Контакт:** зазор между прямоугольниками следов ≤ 0.5 м при скорости сближения ≥ 1.5 м/с за предыдущую секунду; защита от перекрытия в кадре (нижние кромки согласованы по глубине, $|\Delta y_{\text{bottom}}| \leq 0.15 \cdot h$).
- 2. **Шок** в $[-0.3, +0.8]$ с от контакта: Δv за 1 с $\leq -\max(2.5 \text{ м/с}, 0.35 \cdot v_{\text{до}})$; или поворот курса $\geq 25^\circ$ за 0.5 с при $v \geq 3$ м/с вне зон поворота; или боковой скачок ≥ 0.8 м; или стоявший участник внезапно смещается.
- 3. **После удара:** участники стоят (< 0.5 м/с) $\geq 2-3$ с вне обычных зон остановки и не в очереди.
- 4. **Уязвимые:** h/w бокса пешехода или двухколёсника падает на $> 40\%$ за 0.5 с; райдер отделяется; трек человека обрывается под машиной.
- 5. **Один против объекта:** $\Delta v \geq 5$ м/с за ≤ 0.5 с у края дороги, опоры или островка, без других участников в 3 м.
- 6. **Патология треков:** обрыв вдали от края кадра или слияние двух ID в один блок в пределах 2 с от сигналов 1-2.

Границы. START — первый кадр с зазором ≤ 0.2 м (или контактом масок) в окне $[t_c - 1, t_c + 0.5]$ после повторного трекинга на полной частоте; если геометрия ненадёжна — $t_{\text{шок}} - 0.1$ с. END = $\max_i \min(t_{\text{stop},i}, t_{\text{exit},i})$ (формула «min(все остановились, последний уехал)» неверна при смешанных исходах). Цепная авария — один сегмент (склейка < 1 с), min 1.0 с, предохранитель ≤ 3 аварий на видео при $p < 0.9$.

(в) VLM — по умолчанию выключен

Лимит 5 ГБ (Qwen3-VL-2B — 4.26 ГБ в bf16), у T4 нет bf16, запросы съедают бюджет коротких клипов, а точность времени у VLM на CCTV плохая (Molmo-7B zero-shot 0.343 против 0.979 у людей; MAE 1-2 с даёт лишь 0.29-0.58 совпадения для 3-секундного события). Go только если: ограничивает именно точность accident; ≥ 0.8 balanced accuracy на ≥ 40 кандидатах; ≤ 3 с на запрос на T4 без NaN; прирост F1 ≥ 0.05 на proxy-dev. Кандидаты: Qwen3-VL-2B (GGUF Q8 + mmpoj ≈ 2.28 ГБ), Cosmos-Reason2-2B, InternVL3.5-1B. Протокол: 6-12 кадров из $[t-2, t+3]$ с, рамки участников, вопрос Yes/No, max_new_tokens=1.

(б) Кроп-верификатор (подтверждает, никогда не задаёт границы)

Модель	Стоимость	Лицензия
X3D-S / X3D-M (PyTorchVideo)	2.96 / 6.72 GFLOPs на вид, 3.8 М параметров	Apache-2.0; код вендорим (PyPI 0.1.5 сломан с новым torchvision)
VideoMAE V2 distilled ViT-S/16 (vit_s_k710_d1_from_giant.pth)	≈ 22 М, 44.3 МБ; ≈ 57 GFLOPs на клип 16x224	код MIT, HF-репо apache-2.0 (не путать с VideoMAEv2-Base — CC-BY-NC)
VideoMAE V2 ViT-B/16	≈ 87 М, 173.6 МБ	как у ViT-S; апгрейд после рабочего S

- **Вход:** кроп 1.5-2x объединённого бокса пары из 4K-исходника, 160-224 px, 16 кадров (tubelet 2), опционально второй вид — полный кадр 224.
- **Выход:** 3 класса — норма / удар / последствия (опционально near_miss).
- **Позитивы:** окна $[s - 0.5, s + 1.5]$ из ACCIDENT, CADP, TAD, UCF-Crime. **Трудные негативы** — все срабатывания правил на наших сэмплах, которые авариями не являются: самые ценные данные проекта.
- **Обучение:** focal loss, AMP fp16, layer-wise LR decay; калибровка температурой на proxy-dev; веса fp16 safetensors ≈ 44 МБ. Внимание — явный softmax, fp16 на Turing работает.

near_miss — только правила, высокая точность

- 1. Конфликт: TTC ≤ 1.5 с (диски следов + 0.3 м сходятся) или PET ≤ 1.5 с.
- 2. Уклонение в $[t_{\text{minTTC}} - 2, t_{\text{minTTC}} + 0.5]$ с: Δv за 1 с ≤ -3 м/с при $v \geq 4$ м/с; или боковое смещение ≥ 0.8 м за 1.5 с; или резкая остановка/шаг назад пешехода.
- 3. Нет контакта: min зазор > 0.3 м, нет шока, нет кандидата в аварию.
- 4. START — hinge-onset уклонения; END — зазор $\geq \max(2 \text{ м}, 1.5 \cdot \min)$ и растёт, TTC > 3 с; расширение ≈ 0.25 с.
- 5. Исключаем торможение перед красным и сжатие очереди. Публичного CCTV-набора с разметкой near-miss не нашлось — кандидатов (TTC < 2 с) добываем на сэмплах.

Звук (только Part A, узкая роль). LPCM 48 кГц в сэмплах — ≈ 65 МБ на 6.2 ГБ файла. Детектор онсетов без обучения (0.13 с CPU на минуту) подтягивает START визуальной аварии к звуку удара с поправкой на задержку d/343 м/с; сам событие не создаёт. На 106 мин реальных перекрёстков: 32.9 ложных онсетов в час на «сильном» уровне. Вставленные удары находятся в окне ± 1 с в 52/83/91% случаев при 0/+5/+10 дБ; при σ визуального старта 0.5 с доля IoU ≥ 0.7 растёт с 0.845 до 0.930 (модель). Классификаторы AudioSet (PANNs) аварии почти не ловят (6-19%), заносы — 33-53%. Условие включения — звук есть в тесте [орг.] и A/B на dev без потерь.

4.9 Part B: причинный RiskEstimator



Рис. 5. Part B использует только кадры, полученные через step, и собственную перцепцию. Ядро риска принимает список треков, поэтому экашированные треки сэмплов прогоняются офлайн за секунды.

Признак	Определение	Начальные пороги
TTC по капсулам	Первое $t \in [0, 5]$ с (шаг 0.1 с), при котором следы пересекаются, при постоянной скорости	1.5 с — критично, 3.0 с — повышенный
Согласованность TTC	Доля последних 0.8 с, в которых TTC падал $\approx$ на 1 с/с (± 0.4)	Главный фильтр против следования в потоке
MTTC	TTC с учётом ускорения (ограничение $[-8, +3]$ м/с ²)	Только пары в одной полосе
d _{min} и время до неё	На горизонте 5 с	d _{min} < 1.5 м повышает риск
Прогнозные PET и T2	Время прибытия в точку конфликта и её освобождения	≤ 2 с — повышенный, ≤ 1 с — критичный
DRAC	Попутное: $\Delta v^2 / (2 \cdot \text{зазор})$; пересекающееся: $v^2 / (2 \cdot \text{расстояние до точки})$	3.35 м/с ²
Скорость сближения	$-d \parallel \Delta p \parallel / dt$	> 8 м/с — высокая
Одиночные предвестники	Торможение $\Delta v \leq -3$ м/с за 1 с; занос (рыскание > 0.35 рад/с при > 5 м/с); проезд на красный; встреча (> 120° от полосы ≥ 1 с); пешеход на проезжей части при TTC < 4 с; стоящий объект в живой полосе	Причинные версии флагов Part A из общей библиотеки

```
u = -6.0 + 2.5*clip((3.0-TTC)/1.5, 0, 2) + 1.5*clip((DRAC-2.0)/1.35, 0, 2)
    + 1.0*clip((2.0-PETpred)/1.0, 0, 1.5) + 1.0*clip((1.5-dmin)/1.5, 0, 1)
    + 1.0*clip((v_close-3)/5, 0, 1) + 1.0*ttc_consistent + 1.0*violation_flag
    + 0.5*vru_involved + 0.5*evasive_now - 1.5*[both speeds < 2 m/s] - 1.0*min(1, conflict_age/6 s)
r = 1 - prod(1 - sigmoid(u)) по трём самым опасным парам # все пары насытили бы r в плотном потоке
score = 0.5 + 0.4999*r, если тревога включена; иначе min(0.4999*r, 0.4999)
```

Откуда позитивы для калибровки

- 1. Инъекция столкновений в треки нашей сцены: сдвигаем по времени одну из двух реальных траекторий, чтобы они одновременно пришли в точку конфликта (это и есть s); удар сзади — лидер резко тормозит, последователь реагирует на 1.5 с позже; резкая смена курса за 1–2 с до удара. Метки кадров — точно по правилам метрики.
- 2. Клипы ACCIDENT с моментом удара: признаки без масштаба (TTC и PET в секундах, дистанции в длинах бокса). Контекста до удара мало: ≥ 5 с есть лишь у 46% клипов.
- 3. Почти-аварии из сэмплов — прокси-позитивы, но не негативы (Part B их игнорирует).

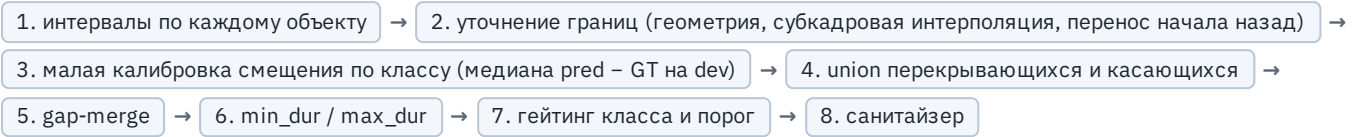
Позже веса учим логистической регрессией (8–12 признаков, L2) или LightGBM с монотонными ограничениями (deterministic=True). Цель ранжирующего канала — «авария в течение 5 с», тревожного — «в течение 10 с». Для AP нужна только монотонность, для F1 — правильное положение порога 0.5.

Формирование тревог

Параметр	Значение
Порог включения	По бюджету ложных: ≤ 1.8/ч на сэмплах вне окон near-miss (≤ 0.5 ложной на тест ≈ 0.28 ч)
Отпускание	Ниже порога минус 0.5 по логиту, задержка 0.5 с
Мин. время включения	Нет — любая задержка сдвигает начало к контакту
Мосты	Для той же пары ≤ 3 с до контакта
Макс. возраст	8 с, без принудительного повторного включения
После контакта	Скор пары ≈ 0.05, подавление этой пары ≤ 10 с
Защита от слияния	Серия, начавшаяся на игнорируемом кадре, не ближе 2 с к предаварийной тревоге
Прогрев	0, пока треки моложе 0.8 с
Ошибка внутри step	try/except : затухающий скор или 0.0, никогда не держать ≥ 0.5

Инженерия Part B: `torch.use_deterministic_algorithms(True, warn_only=True)` (иначе операция без детерминированного ядра бросит исключение посреди видео); `stride` из `meta` ($\approx 10\text{--}15$ Гц); модули `perception.py`, `kinematics.py`, `conflicts.py` (чистый numpy, юнит-тесты), `risk_model.py`, `alarm.py`. **Тест причинности в CI:** во время стриминга подменить `cv2.VideoCapture` и `open` и убедиться, что `RiskEstimator` не открывает файл. Альтернативы на будущее: причинное окно VideoMAEv2-S, заканчивающееся в `t` (рецепт TimeSouth, реализовать самим), или MoViNet-A0 Stream.

4.10 Постобработка сегментов



Санитайзер (последний шаг, покрыт юнит-тестами)

- 1. метка из `CLASSES`;
- 2. Python `float`, отбросить NaN и бесконечности;
- 3. `start = max(0, start)`;
- 4. `end - start ≥ max(min_dur_c, 0.05)`;
- 5. округлить до 3 знаков;
- 6. `end = min(end, floor(duration, 3))`, `duration = CAP_PROP_FRAME_COUNT / fps`;
- 7. повторно проверить `start < end`;
- 8. объединить перекрытия одного класса; отсортировать.

Событие за концом видео получает `end = duration`, событие с первого кадра — `start = 0`.

Класс	gap, c	min, c	max, c
accident	1.0	1.0	30
near_miss	0.5–1	0.8	10
red_light	0.5	0.5–0.8	15
wrong_way	2	1.5–2	—
illegal_u_turn / illegal_turn	1	1.5–2	20–30
stopped_vehicle	2–5	10	—
jaywalking	1	1–1.2	—
failure_to_yield	0.5	0.4–0.5	10
solid_line_crossing	0.5	0.5	8
stop_line	2	1–3	150
congestion	10–15	20–30	—
road_obstacle	5	1 / 3–5	—
fire_smoke	5–10	3–5	—

Настройка. Каждое изменение прогоняем через официальный `evaluate.py --gt dev_gt.json` и смотрим F1 по каждому т отдельно: провал именно при 0.7 — проблема границ, а не детекции. Схема leave-one-video-out: настраиваем на трёх видео, предсказываем четвёртое, объединяем отложенные предсказания и считаем один общий скор. 2–4 физически осмысленных параметра на класс, грубая сетка ≈ 5 значений, берём середину плато; порядок: `congestion` → `stop_line` / `red_light` → `stopped_vehicle` → `accident` → `near_miss`. Строка чувствительности: сдвиг границ на $\pm 0.5, \pm 1, \pm 2$ с и F1@0.7. Для классов, которых нет в сэмплах, пороги — из физики и синтетических тестов, без подгонки.

4.11 Итоговый стек и бюджет весов

Слой	Основной	Запасной
Декод	OpenCV <code>CAP_FFMPEG</code> , <code>grab/retrieve</code>	PyAV 17.1.0 после бенчмарка и проверки совпадения
Детектор Part A	YOLO26m (дообученный), вход 1280, ROI дороги, ≈ 15 Гц	RF-DETR-S/M (Apache-2.0)
Детектор Part B	YOLO26s@960 (n@640 на T1–T2), $\approx 10\text{--}15$ Гц	RF-DETR-N
Трекер	ByteTrack без ReID и GMC + офлайн-сшивки в Part A	OC-SORT
Светофор	ROI ламп, относительная яркость, <code>max-pool</code> против мерцания, <code>Viterbi</code>	Autoware classifier / маленькая CNN на кропах 32×96
Огонь и дым	YOLO26n/s на D-Fire (CC0) + негативы сцены, 1–2 Гц	готовый <code>rabahdev/fire-smoke-yolov8n</code> на день 1
Препятствия	Двойной фон + YOLOE-26 с запечённым словарём	OWLv2 / MM-Grounding-DINO tiny на кропах
Аварии	Кинематика + (опц.) X3D или VideoMAEv2-S на кропе	V-JEPA 2.1 ViT-B + probe как второе мнение
Поток (congestion)	OpenCV DIS optical flow @960×540	RAFT small (torchvision, 4 МБ)
Инференс	PyTorch FP16, веса по локальным путям	PyTorch CPU (YOLO26n, больший stride)

Компонент весов	Выбор	Размер
Детектор	дообученный YOLO26m <code>best.pt</code> (s — при дефиците времени)	≈ 0.044 ГБ

Компонент весов	Выбор	Размер
Трекер, зоны, HSV-светофор, MOG2, DIS	—	0
<code>fire_smoke</code>	YOLO26s на D-Fire	≈0.020 ГБ
<code>road_obstacle</code>	YOLOE-26l-seg (или s) с запечённым словарём	0.079 (0.031) ГБ
Светофор (фолбэк)	Autoware classifier ONNX	0.009 ГБ
accident / near_miss	VideoMAEv2-S fp16 (ViT-B: 0.174 ГБ)	0.044 ГБ
Опц. второе мнение / верификатор кропов	V-JEPA 2.1 ViT-B + probe / MM-GDINO-T или OWLv2	≈0.22 / 0.69 ГБ
Опц. VLM	Qwen3-VL-2B GGUF Q8_0 + mmproj Q8 / MiniCPM-V-4.6 GGUF	2.28 / 1.64 ГБ
Ядро ≈0.15–0.21 ГБ (.pt); + open-vocabulary верификатор ≈1.1 ГБ; + VLM ≈3.4 ГБ — остаётся > 1.5 ГБ запаса до лимита 5 ГБ		

4.12 Инженерия: установка, офлайн, детерминизм, репозиторий

Рубрика Code: 40% — «запускается как сдано», 25% — воспроизводимость. **Основной путь — rip, запасной — протестированный Dockerfile**; оба проверяются на каждом кандидате в релиз.

```
# requirements.txt: Linux x86_64, Python 3.10–3.13, драйвер NVIDIA >= 525 (эскиз; пин OpenCV — после замера, §3.5)
--extra-index-url https://download.pytorch.org/whl/cu126
torch==2.14.0+cu126 # cu126 работает с драйвером >= 525; CUDA 13 с PyPI требует >= 580
torchvision==0.29.0+cu126
ultralytics-opencv-headless==8.4.162 # HE "ultralytics": тот тянет opencv-python и libGL
opencv-python-headless==4.13.0.92 # без пина встанет 5.0.0.93, на ней ничего не тестировали
numpy==2.2.6 ; python_version < "3.11"
numpy==2.3.5 ; python_version >= "3.11"
scipy==1.15.3 ; python_version < "3.11"
scipy==1.16.3 ; python_version >= "3.11"
lap==0.5.13 # без него трекер Ultralytics ставит пакет сам и падает офлайн
PyYAML==6.0.3
# av==17.1.0 # только если PyAV пройдёт бенчмарк (последняя версия с ср310)
```

Офлайн-режим

`src/<pkg>/env.py` — первый импорт в `solution.py`; переменные ставим принудительно, до импорта `ultralytics/torch/transformers`:

- `YOLO_OFFLINE=1`, `YOLO_AUTOINSTALL=false`, `YOLO_CONFIG_DIR` в `/tmp`;
- `HF_HUB_OFFLINE=1`, `TRANSFORMERS_OFFLINE=1`, `HF_HUB_DISABLE_TELEMETRY=1`;
- `HF_HOME`, `TORCH_HOME` — внутрь `weights/`; `MPLBACKEND=Agg`;
- `settings.update({"sync": False})` в Ultralytics.

Что может неявно полезть в сеть: веса Ultralytics по имени (всегда абсолютный путь через `Path(__file__)`), шрифт Arial при рисовании, автоустановка `lap`, CLIP для YOLOE при `set_classes()` (промпты запекаем при экспорте), RF-DETR без локального чекпойнта, `torch.hub/pretrained=True`. Проверка: `netguard/sitecustomize.py` (подмена `socket.connect`, `getaddrinfo`) и `docker run --network none`.

Весы

В git — файлы < 50 МБ; крупнее — GitHub Release assets (< 2 ГБ на файл). `download.sh` идиомпотентный, со сверкой SHA-256 и зеркалом на HF. **Git LFS не использовать** (клоны жюри тратят квоту). При импорте проверяем, что веса на месте; если нет — внятная ошибка и пустые предсказания, а не падение.

Детерминизм

- Сиды `random`, `numpy`, `torch`; `cudnn.benchmark=False`, `cudnn.deterministic=True`, `use_deterministic_algorithms(True, warn_only=True)` (в CI `warn_only=False`).
- Только последовательный декод, фиксированный батч, `sorted()` везде, где порядок влияет на жадное сопоставление.
- Никаких решений по настенным часам, кроме аварийного тормоза; гистерезис и запас у каждого порога против FP16-шума.
- `tools/compare_predictions.py run1.json run2.json` с допусками `1e-3` с и `1e-4` на риск, блок `log` игнорируется.
- `predictions_samples.json` генерируется на T4 из тегированного коммита; в README — GPU, драйвер, версия `torch`.

Устойчивость

- Каждый этап в своём `try/except` с накоплением частичных результатов; при `OutOfMemoryError` — `empty_cache()` и вдвое меньший батч.
- Проверка здоровья CUDA перед каждым видео («липкая» ошибка отравляет все следующие видео).
- Segfault, OOM-kill, зависание теряют весь прогон: кольцевые буферы фиксированного размера, рискованные этапы — в подпроцесс с таймаутом.
- Fuzz: 1 кадр, 0.5 с, битый файл, портрет, 30/переменный fps, 10 мин, без звука.

```
solution.py                # тонкий: делегирует в src/trafficwatch
run_submission.py, evaluate.py  # без изменений; хэш сверяется в CI
requirements.txt, constraints-py310.txt, Dockerfile
weights/ download.sh SHA256SUMS scene/
configs/ scene.yaml pipeline.yaml tracker.yaml
src/trafficwatch/
  env.py
  runtime/{budget, safety, sanitize, logging}.py
  io/video.py                # время = idx/fps
  perception/{detector, tracker_config, tracker}.py
  scene/{geometry, registration, signal_state, static_engine}.py
  features/{kinematics, conflicts, flowfield}.py
  rules/<модуль на класс>.py
  events/postprocess.py
  audio/onsets.py            # опц., только Part A
  risk/{estimator, alarm, risk_model}.py
  viz/{render, timeline}.py  # общее с сайтом
tools/ labels_to_gt.py compare_predictions.py check_proxy.py measure_alignment.py budget.py
tests/ test_rules_synthetic.py test_sanitize.py test_causality.py test_format.py test_determinism.py
training/ notebooks/ (только EDA) dev_labels/
```

CI (GitHub Actions): ruff, юнит-тесты, CPU smoke-тест на синтетическом клипе 5 с под netguard, затем `evaluate.py --validate-only --gt dev_gt.json`.

Ловушки Windows: `.gitattributes` с `*.sh text eol=lf`; `git update-index --chmod=+x weights/download.sh`; регистр путей; явный `cv2.CAP_FFMPEG`. Docker без `--gpus` молча работает на CPU — код обязан это пережить (план для CPU, YOLO26n, большой stride).

Репетиция перед тегом	Что доказывает
<code>pip install</code> в чистых <code>python:3.10-slim</code> и <code>3.12-slim</code>	Зависимости разрешаются, нет компиляции и <code>libGL</code>
Чистый клон тега → <code>venv</code> → <code>download.sh</code> → сэмплы без сети (WSL2 + <code>--network none</code>)	Настоящий офлайн, регистр путей, CRLF
То же на T4 (Kaggle или Colab)	Время против 3х, поведение FP16
То же на <code>g4dn.2xlarge spot</code> (желательно)	CPU-сторона бюджета
Два прогона + <code>compare_predictions.py</code>	Детерминизм
<code>evaluate.py --validate-only --gt dev_gt.json</code>	Формат, длительности, все видео на месте

5. Все 14 классов: конвенции и методы

5.1 Официальные определения и границы

Колонки «Начало» и «Конец» — конвенции, которым следовали аннотаторы организаторов (перевод из PDF задания). Именно они определяют, куда ставить границы и в нашей разметке, и в предсказаниях.

id	Определение	Начало (start)	Конец (end)
accident	Контакт двух и более участников движения или участника с неподвижным объектом	Первый кадр, где виден контакт	Все участники остановились или покинули кадр
near_miss	Резкое торможение или манёвр, чтобы избежать столкновения; контакта нет	Начало уклоняющегося действия	Участники разъехались
red_light	ТС пересекает стоп-линию, когда его сигнал красный	Перёд ТС пересекает стоп-линию	ТС покидает перекрёсток или кадр
wrong_way	ТС движется против направления своей полосы, включая езду по встречной	ТС въезжает на встречную полосу	ТС вернулось в правильную полосу или покинуло кадр
illegal_u_turn	Разворот там, где его запрещают разметка или знаки	ТС начинает поворот	ТС завершило разворот
stopped_vehicle	ТС стоит на проезжей части 10 с и дольше, не в очереди у светофора	ТС останавливается	ТС снова поехало или убрано
jaywalking	Пешеход на проезжей части вне перехода	Пешеход ступил на дорогу	Пешеход покинул дорогу
failure_to_yield	ТС проезжает переход, пока на нём пешеход или он на него ступает	ТС въезжает на переход	ТС покидает переход
illegal_turn	Поворот не из той полосы или в запрещённом направлении	ТС начинает поворот	ТС завершило поворот
solid_line_crossing	Перестроение или манёвр через сплошную разметку	Колесо пересекает линию	ТС полностью в новой полосе
stop_line	ТС останавливается за стоп-линией на красный, не въезжая на перекрёсток	ТС останавливается	Загорается зелёный
congestion	Движение стоит или ползёт по всем полосам одного направления	Очередь перестаёт двигаться	Очередь рассосалась
road_obstacle	Обломки, животное или упавший предмет на проезжей части	Препятствие появляется	Препятствие убрано
fire_smoke	Видимый огонь или дым от ТС или на дороге	Первый видимый дым	Дым рассеялся или кадр закончился

Общие правила из PDF и FAQ

- Одно событие — один непрерывный сегмент одного класса. Два одновременных события разных классов — две записи (встречка, вызвавшая аварию, — это `wrong_way` + `accident`).
- Сегменты одного класса не должны перекрываться. Если два события одного класса происходят одновременно, их выдают одним сегментом, покрывающим оба; организаторы размечают так же.
- Если событие продолжается за концом видео, `end_sec` = длительность видео.
- Превышения скорости в списке нет: без калибровки его не измерить. Все 14 классов официальные, любой может быть в тесте, и в тесте есть события, которых нет в сэмплах.

5.2 Как мы детектируем каждый класс

Класс	Сигнал и метод	Опора для границ	Сложн.	Приоритет
stopped_vehicle	Трек стоит ($< 0.3\text{--}0.5$ м/с, смещение $< 0.7\text{--}1$ м за 2 с) ≥ 10 с на проезжей части, не в очереди у сигнала $\geq 50\%$ времени; ghost-боксы при перекрытии; статичный движок	Старт = момент остановки (перенос назад), конец = устойчивое движение ≥ 1 с или «фон вернулся»	2	P1, вкл.

Класс	Сигнал и метод	Опора для границ	Сложн.	Приоритет
jaywalking	Ступни в зоне «проезжая часть – (переходы + 1 м) – островки – тротуары»; отступ $\max(0.4 \text{ м}, 0.1 \cdot \text{рост})$; гистерезис 0.6 с; райдеры исключены; регрессия роста по строке	Интерполированные пересечения нуля расстоянием до бордюра	2	P1, вкл.
wrong_way	Курс против полосы (< -0.5) и против поля направлений (< -0.5), скорость $> 1.5 \text{ м/с}$, $\geq 1.5 \text{ с}$ и $\geq 8 \text{ м}$; защита от смены ID; внутри перекрёстка не срабатывает	Пересечение разделителя или первая детекция → возврат или выезд из кадра	2	P1, вкл. [camera.md]
congestion	По направлению: все полосы с пиксельной занятостью ≥ 0.3 и скоростью $\leq 2 \text{ м/с}$ (медиана треков, при < 3 треках – DIS-поток); медиана 5 с; гистерезис; объединение направлений в одну шкалу	Остановка очереди (перенос назад) → скорость $> 4 \text{ м/с}$ или занятость < 0.15 в течение 10 с	3	P1–P2, H1
accident	Контакт пары на плоскости земли + шок (Δv , рыскание) + неподвижность после; удар об объект; кроп-верификатор (§4.8)	Первый видимый контакт → $\max; \min(t_{\text{stop}}, t_{\text{exit}})$	5	P2, вкл. точность
near_miss	Конфликт ($\text{TTC} \leq 1.5$ или $\text{PET} \leq 1.5 \text{ с}$) + уклонение ($\Delta v \leq -3 \text{ м/с}$ за 1 с или смещение $\geq 0.8 \text{ м}$) + нет контакта	Hinge-onset уклонения → разъезд	5	P3, при гейте
red_light	Передняя точка пересекает стоп-линию на красный (фаза по ROI + Viterbi); $t_c - \text{red_onset} \geq 0.3 \text{ с}$, скорость $> 1 \text{ м/с}$, курс в 45° от подхода; направо на красный – только при знаке 5.42	Субкадровое пересечение фронтом → выезд из перекрёстка или кадра	3 / 4–5	P3, если сигнал виден
stop_line	Стоит $\geq 1 \text{ с}$, перед линией $> 0.3 \text{ м}$, в перекрёсток не въехал, красный; только ближняя и средняя зона (rrtm); несколько машин в одном красном – один сегмент	Остановка → включение зелёного	3–4	P3, если фаза известна
failure_to_yield	ТС $> 1 \text{ м/с}$ проезжает переход, пока пешеход на нём ($\leq 6 \text{ м}$ от траектории) или шагает на него ($> 0.5 \text{ м/с}$ к дороге) в окне $[t_{\text{in}} - 0.5, t_{\text{out}}]$	Перед въехал → зад выехал (модель перед/зад обязательна)	3	P2, если переход в кадре
solid_line_crossing	Знаковое расстояние скорректированного следа (нижние углы бокса) до сплошной; полный переход; удержание $\geq 1 \text{ с}$ и отклонение $\geq 0.6 \text{ м}$; для «сплошная–прерывистая» – только со стороны сплошной	Колесо на линии ($ d = w/2$) → полностью в новой полосе	3	P2, если есть сплошные
illegal_turn	Манёвр (пара въезд–выезд) не разрешён для полосы у стоп-линии; illegal_u_turn его исключает	Рыскание $> 5^\circ/\text{с}$ при курсе в 15° от въезда → курс в $15\text{--}20^\circ$ от выезда	4	P4, при запретах
illegal_u_turn	$\Delta\theta \geq 150^\circ$ за $\leq 20 \text{ с}$, полосы до и после встречные, вершина дуги в зоне запрета, скорость $> 1 \text{ м/с}$ на дуге (отсекает развороты в три приёма)	По скорости рыскания	3	P4, при запрете
road_obstacle	(а) животные COCO на проезжей части $\geq 1 \text{ с}$; (б) статичный блоб двойного фона + верификатор YOLOE/OWLv2	Появление (поиск назад по кадрам) → удаление	4	P5, выкл.
fire_smoke	YOLO26 на D-Fire + негативы сцены; 7 из 10 отсчётов за 5 с; не привязан к едущей машине; мерцание или рост	Первое проявление (порог 0.15, назад) → исчезновение или конец видео	3	P5, выкл.

Детали по ключевым классам

stopped_vehicle

```
stat = (v < 0.5) & (disp_2s < 1.0)      #  
метры, через гомографию  
runs(stat, fill_gap=1-3 с, min_dur=10 с)  
на проезжей части, не в parking_bay  
не in_signal_queue ≥ 50% прогона
```

`in_signal_queue`: машина в зоне очереди подхода, фаза красная/жёлтая (или < 8 с с начала зелёного), и она у стоп-линии (< 3 м) или за стоящей машиной из очереди. **Старт — момент остановки, а не +10 с**: GT [0, 25] против выдачи [10, 25] даёт IoU 0.6 и провал при $\tau = 0.7$. После аварии `stopped_vehicle` выдаём (классы могут перекрываться). Автобусы на остановке и такси — по буквальному определению, парковочные карманы исключаются [орг.]. Защита от автоэкспозиции: если > 40% дороги меняется за 1 с — переинициализировать фон и заглушить статичные события на 10 с.

jaywalking

Вход: ступни внутри дороги на $d \geq \max(0.4 \text{ м}, 0.1 \cdot \text{рост})$ в течение 0.6 с; выход: $d < 0$ в течение 0.6 с; **затем границы переносятся назад к пересечениям нуля** (иначе старт опаздывает почти на 1 с). Min 1–1.2 с, склейка ≤ 1 с. Спорно [орг.]: люди, выходящие из машины, полиция, дорожные рабочие. Перекрёсток целиком не исключать: там бывает диагональное пересечение.

congestion : две гипотезы

H1 (по умолчанию): очередь у сигнала засчитывается, только если она не рассасывается на зелёном ≥ 10 с, или доходит до края зоны или в перекрёсток, или длится > 1.5× самого длинного красного из EDA (без сигнала — 45 с). **H2 (переключатель)**: любая остановка всех полос ≥ 20 с. Dev размечаем в обоих вариантах [орг.]. Min 20–30 с, склейка 10–15 с.

wrong_way

Задний ход и парковку отсекают условия «скорость ≤ 2 м/с и путь < 8 м». На дуге разворота выравнивание ≈ 0 , поэтому класс срабатывает, только если машина продолжает ехать по встречной. Сочетания: обгон через сплошную = `solid_line_crossing` + `wrong_way`; поворот на одностороннюю навстречу = `illegal_turn` + `wrong_way`.

red_light против stop_line

Машина, заехавшая за линию на красный и остановившаяся до перекрёстка, — по умолчанию только `stop_line` [орг.]. Если фаза выводится из потока, дополнительно требуем ≥ 2 пересечений на конфликтующем подходе за ± 3 с и стоящую собственную очередь. Проезд на жёлтый — не `red_light`: метки ставим по определениям задания, а не по ПДД.

Синтетические тесты траекторий

Для каждого класса вручную заданный трек в координатах плоскости земли должен давать ровно один сегмент с ожидаемыми границами. Это единственная проверка для классов, которых нет в сэмплах.

5.3 Какие классы включаем

Набор	Классы	Условие
включены по умолчанию	<code>stopped_vehicle</code> , <code>jaywalking</code> , <code>wrong_way</code> , <code>congestion</code> (H1), <code>accident</code>	Для <code>wrong_way</code> нужны направления полос [camera.md], иначе — самокалиброванное поле направлений. <code>accident</code> включён, потому что наличие Part B делает аварии в тесте вероятными, но с приоритетом точности.
только если сцена поддерживает	<code>red_light</code> , <code>stop_line</code> , <code>failure_to_yield</code> , <code>solid_line_crossing</code> , <code>illegal_turn</code> , <code>illegal_u_turn</code> , <code>near_miss</code>	Виден светофор или фаза выводится надёжно; есть переход; есть сплошные; есть запреты. Если предпосылки нет, $f \approx 0$ — класс выключаем (можно просто убрать из CLASSES). Правило игнорирования кадров вокруг near-miss в Part B не означает, что <code>near_miss</code> есть в тесте.
редкие, выключены	<code>fire_smoke</code> , <code>road_obstacle</code>	Только при многосигнальном подтверждении, проверке на внешнем банке негативов и очень высокой точности.

Окончательное решение по каждому классу принимается по правилу $p \cdot f > (1-p) \cdot q \cdot \bar{f}$ с частотой ложных срабатываний в час, измеренной на сэмплах, — после того как появится camera.md и dev-разметка.

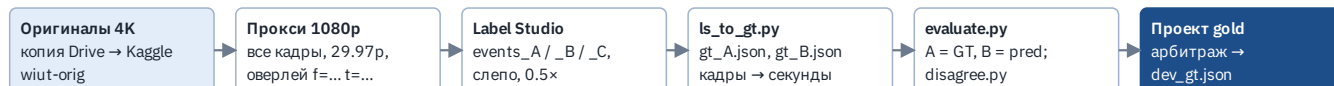
6. Разметка данных

Меток для сэмплов организаторы не дают, а «без dev-набора вы гадаете» — это их собственный совет. Своя разметка — самые ценные данные проекта: на ней настраиваются все пороги, считается согласие людей (реальный потолок F1 при $\tau = 0.7$) и строятся страницы результатов на сайте.

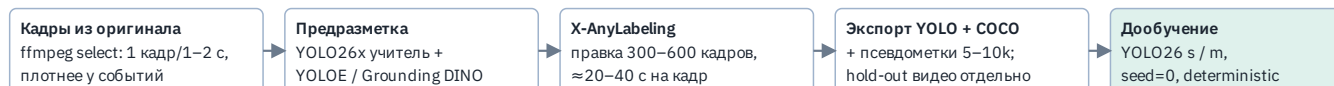
6.1 Три задачи разметки и инструменты

Задача	Что размечаем	Основной инструмент	Запасной	Выход
1. События	Временные сегменты 14 классов с точностью до кадра; классы могут перекрываться	Label Studio 1.23.0 (Video + TimelineLabels), Apache-2.0	ELAN 7.1; BORIS 9.15; VIA 3	JSON-экспорт → <code>ls_to_gt.py</code> → <code>ground_truth.json</code>
2. Боксы и треки	ТС, двухколёсные, пешеходы, светофоры, препятствия, огонь и дым для дообучения детектора	X-AnyLabeling 4.0.6 с предразметкой (YOLO, YOLOE, Grounding DINO, SAM), GPL-3.0	CVAT v2.76 self-hosted	YOLO / COCO (слияние — Datumaro)
3. Геометрия сцены	Проезжая часть, полосы и направления, стоп-линии, переходы, сплошные, ROI светофоров, зоны запретов и очередей	labelme 7.7.0 на 4K PNG, GPL-3.0 (JSON наш)	CVAT; VIA; makesense	<code>labelme_to_scene.py</code> → <code>scene.json</code> → <code>scene.yaml</code>

1. События



2. Боксы



3. Геометрия

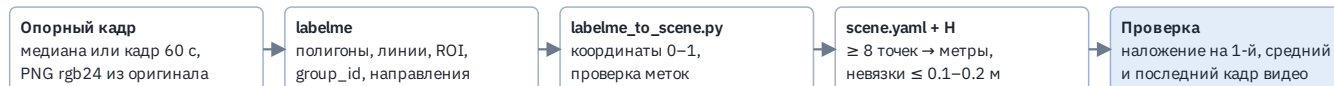


Рис. 6. Три потока разметки. Всё работает локально или на своём сервере: футаж организаторов в облачные сервисы разметки не загружаем.

Роли: **A** — пайплайн (прокси, хост Label Studio, конвертеры, прогоны `evaluate.py`; тоже размечает); **B** — владелец гайдлайна и первый арбитр (правила и лист примеров; тоже размечает); **C** — геометрия и боксы (labelme и X-AnyLabeling; размечает события на 1–2 видео).

6.2 Подготовка видео: прокси с сохранением кадров

Браузеры, Windows Media Foundation и JavaFX оригиналы 4:2:2 10-bit не воспроизводят (Chrome и Edge играют программно, но в 4K теряют кадры, а переход занимает 0.5–3.2 с). Поэтому все браузерные и десктопные плееры работают с **прокси**, в котором сохранён каждый кадр и частота 29.97. Проверено на синтетике 3840×2160 yuv422p10le 30000/1001: 120 пакетов на входе и выходе, `start_time` 0, длительности совпадают.

Шаг 0. Сравнить длительности аудио и видео в каждом файле (Label Studio предупреждает, что разные длительности дают лишние кадры). Если аудио отличается — заменить аудио-часть команды на `-an`.

```
ffprobe -v error -show_entries stream=index,codec_type,duration -show_entries format=duration -of compact=p=0 C3896.MP4
```

Шаг 1. Прокси для разметки (1080p, оверлей с 0-based номером кадра исходника и pts; ключевой кадр каждые 0.5 с, без B-кадров — быстрая и точная перемотка):

```
ffmpeg -hide_banner -y -i C3896.MP4 -map 0:v:0 -map "0:a:0?" \
-vf "scale=1920:-2:flags=bicubic,format=yuv420p,drawtext=fontfile='C:/Windows/Fonts/consola.ttf':text='f=%{frame_num} t=%{pts}\:flt':x=20:y=20:fontsize=32:fontcolor=white:box=1:boxcolor=black@0.6" \
-fps_mode passthrough -c:v libx264 -preset veryfast -crf 20 -profile:v high -g 15 -bf 0 \
-c:a aac -b:a 128k -movflags +faststart proxy/C3896_p1080.mp4
```

Чистые прокси для EDA, сайта и визуального ревью (без оверлея; 720p ≤ 2.5 Мбит/с для плеера на сайте):

```
ffmpeg -i C3896.MP4 -map 0:v:0 -an -vf "scale=1920:-2:flags=bicubic,format=yuv420p" -c:v libx264 -preset medium -crf 18 \
-g 30 -keyint_min 30 -sc_threshold 0 -bf 0 -fps_mode passthrough -video_track_timescale 30000 -movflags +faststart
C3896_1080p.mp4
ffmpeg -i C3896.MP4 -map 0:v:0 -an -vf "scale=1280:-2:flags=bicubic,format=yuv420p" -c:v libx264 -preset slow -crf 23 -maxrate 2.5M \
-bufsize 5M -profile:v high -level:v 4.0 -g 30 -keyint_min 30 -sc_threshold 0 -bf 2 -fps_mode passthrough \
-video_track_timescale 30000 -movflags +faststart C3896_720p.mp4
```

Чего никогда не делать

- Не добавлять `-r`, фильтр `fps=` или `-ss` перед `-i` — меняется число кадров. Команду из документации Label Studio с `-r 30` не копировать.
- Не выставять цветовые теги вручную: принудительный `bt709` сдвигает BGR в OpenCV в среднем на 6.5–11.8 уровня.
- Ловушка `drawtext: %ipts:flt:3}` добавляет смещение 3 с, а не задаёт точность.
- Цветовые пороги (лампы светофора) настраивать на кадрах, декодированных как в харнесе, а не на прокси.

Шаг 2. Проверить каждый прокси

`nb_read_packets, r_frame_rate = 30000/1001` и `start_time` оригинала и прокси должны совпасть (`-count_packets` читает только контейнер — быстро даже на 6 ГБ):

```
ffprobe -v error -select_streams v:0 -count_packets \
-show_entries
stream=nb_read_packets,r_frame_rate,start_time \
-of csv=p=0 C3896.MP4
```

`check_proxy.py` ORIG PROXY дополнительно сверяет `CAP_PROP_FRAME_COUNT/FPS`, пок кадровое выравнивание (лаг 0), цвет (≤ 3 уровня) и случайные переходы; на неправильных прокси (`-r 25`, `-ss`, `bt709`) проверка проваливается. Скорость: 1080p-прокси $\approx 1.5\times$ реального времени на ноутбуке (≈ 8.5 мин на сэмпл).

6.3 События в Label Studio: как это выглядит

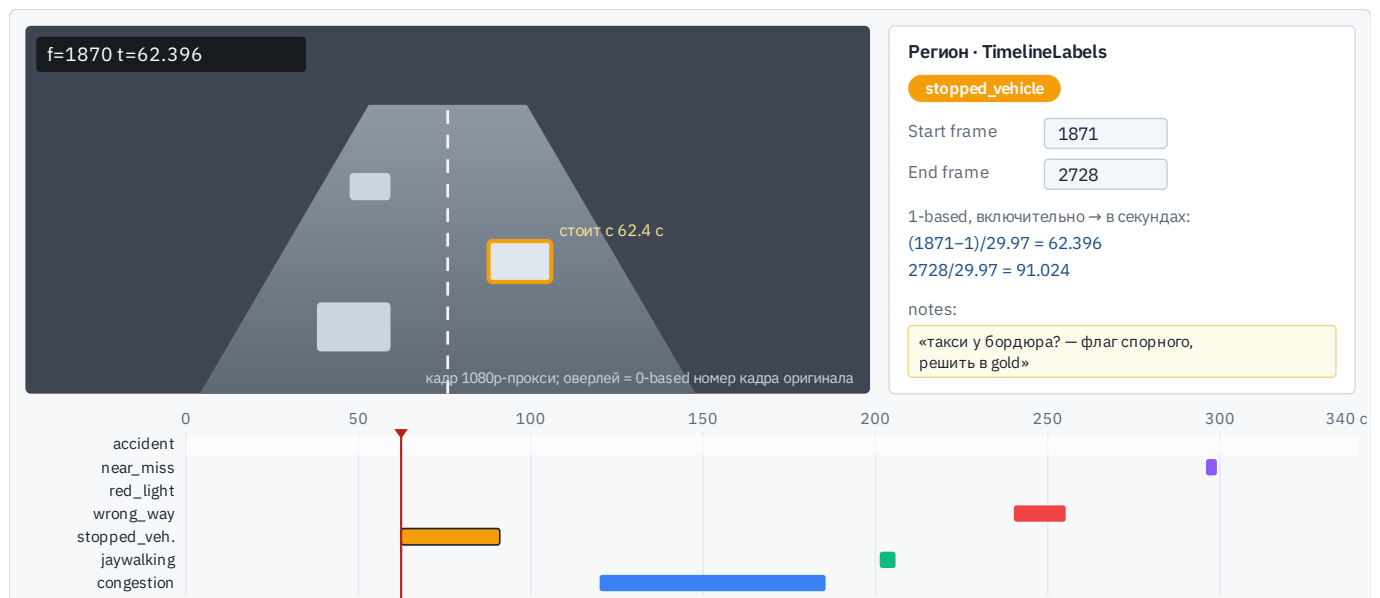


Рис. 7. Макет рабочего места аннотатора: плеер прокси с оверлеем номера кадра, панель региона с полями «Start frame / End frame» и таймлайн по классам (строк 14, здесь показаны 7). Иллюстрация, события вымышленные.

Запуск (один ноутбук-хост; Windows `cmd.exe`):

```
cd /d C:\westhack
uv venv --python 3.12 lsenv
lsenv\Scripts\activate
uv pip install label-studio
set LABEL_STUDIO_LOCAL_FILES_SERVING_ENABLED=true
set LABEL_STUDIO_LOCAL_FILES_DOCUMENT_ROOT=C:\\westhack
label-studio start -p 8080
```

Сокомандники открывают `http://<LAN-IP>:8080`. Хранилище: Settings → Cloud Storage → Local files; URL задач вида `/data/local-files/?d=proxy/C3896_p1080.mp4`. Альтернатива — Docker `heartexlabs/label-studio:latest`.

Задачи для импорта — `fps` и `duration` берём из оригинала, ключ `GT` — имя оригинала:

```
[{"data": {"video": "/data/local-files/?d=proxy/C3896_p1080.mp4",
            "orig": "C3896.MP4",
            "fps": 29.97002997, "duration": 340.34}}]
```

Конфиг разметки (14 классов с горячими клавишами):

```
<View>
  <Video name="video" value="$video"
    frameRate="$fps"
      minPlaybackSpeed="0.25" height="620"
    timelineHeight="320"/>
  <TimelineLabels name="events" toName="video">
    <Label value="accident" hotkey="1"/> <Label
    value="near_miss" hotkey="2"/>
    <Label value="red_light" hotkey="3"/> <Label
    value="wrong_way" hotkey="4"/>
    <Label value="illegal_u_turn" hotkey="5"/>
    <Label value="stopped_vehicle" hotkey="6"/>
    <Label value="jaywalking" hotkey="7"/> <Label
    value="failure_to_yield" hotkey="8"/>
    <Label value="illegal_turn" hotkey="9"/> <Label
    value="solid_line_crossing" hotkey="0"/>
    <Label value="stop_line" hotkey="q"/> <Label
    value="congestion" hotkey="w"/>
    <Label value="road_obstacle" hotkey="e"/>
    <Label value="fire_smoke" hotkey="r"/>
  </TimelineLabels>
  <TextArea name="notes" toName="video"
    placeholder="сомнительные события /
    комментарии"/>
</View>
```

Настройки, без которых метки съедут

- **frameRate по умолчанию 24.** Задаём `frameRate="$fps"` и `fps = 29.97002997` (30000/1001) в данных задачи; для других файлов `fps` берём из `ffprobe` и никогда не хардкодим.
- Диапазоны `TimelineLabels 1-based` и **включительные**: `start_sec = (start - 1)/fps`, `end_sec = end/fps`. Номер кадра в LS = номер на оверлее + 1.
- Нижняя граница скорости в исходниках 0.25, в документации — 1: явно указываем `minPlaybackSpeed="0.25"`, смотрим на 0.5x, у границ шагаем по кадрам и правим поля «Start frame / End frame».
- Слепая двойная разметка: отдельный проект на аннотатора (`events_A`, `events_B`, `events_C`) плюс проект `gold` для арбитража. Экспорт — **JSON**, не **JSON_MIN**.
- В своём браузерном плеере переход к кадру `i` — через `currentTime = (i + 0.5)/fps`: при `i/fps` в 16 случаях из 20 показывается кадр `i-1`. Спорные границы смотрим в `mpv` на оригинале (`.` и `,` — шаг на кадр).

6.4 Формат результата: ground_truth.json

Конвертация экспорта

(tools/annotation/ls_to_gt.py, уже в репозитории):

```
python tools/annotation/ls_to_gt.py
export_events_A.json gt_A.json
python tools/annotation/ls_to_gt.py
export_events_B.json gt_B.json
# --user ID_или_email — фильтр по аннотатору
# --all — взять все аннотации задачи, а не
последнюю
```

- По умолчанию берётся одна, самая свежая аннотация задачи, чтобы случайно не сложить разметку двух людей в один GT.
- fps в GT округляется до 2 знаков (29.97), как в примере организаторов; секунды считаются по точному $30000/1001$.
- duration — как в харнесе: $CAP_PROP_FRAME_COUNT / CAP_PROP_FPS$ (C3896: $10\ 200 / 29.97003 = 340.34$ с); метки проходят через тот же санитайзер, что и предсказания.
- Время — всегда $frame_index / fps$, кадр 0 = 0.0 с. При допущении «25 fps» все времена растянутся в 1.2 раза; при «ровно 30 fps» к 340-й секунде наберёт 0.34 с.

Как выглядит итоговый файл (пример, события вымышленные):

```
{
  "C3896.MP4": {
    "duration": 340.34,
    "fps": 29.97,
    "events": [
      [62.396, 91.024, "stopped_vehicle"],
      [119.987, 185.619, "congestion"],
      [201.301, 205.906, "jaywalking"],
      [240.24, 255.255, "wrong_way"]
    ]
  },
  "C3897.MP4": {"duration": 317.817, "fps": 29.97,
    "events": []}
}
```

Пример пересчёта: кадры 361..570 в LS (на оверлее $f=360..569$) → [12.012, 19.019, "accident"].

6.5 Гайдлайн по границам

Файл docs/annotation_guide.md: конвенции начала и конца дословно из PDF, командные уточнения и лист примеров. **Единообразие конвенции важнее точности до одного кадра.** Если организаторы опубликуют уточнения или примеры GT, они важнее всего, что ниже.

- Кадры и секунды.** Время считаем по оригиналу. *start* — первый кадр, на котором определяющее условие видно выполненным; *end* — последний кадр, на котором оно ещё выполнено (включительно). Конвертер пишет $start_sec = f_start/fps$, $end_sec = (f_end + 1)/fps$.
- Допуск скорера:** при сдвиге δ IoU = $(L - \delta)/(L + \delta)$; для $IoU \geq 0.7$ нужно $\delta \leq 0.18 \cdot L$: у события 3 с это ≈ 16 кадров, у события 1 с — только ≈ 5 . Больше всего внимания — коротким классам: `near_miss`, `red_light`, `solid_line_crossing`.
- Одно событие = один участник (или одна взаимодействующая пара) и один класс.** Две машины, проехавшие на один красный, — два события на уровне разметки, но в GT одновременные события одного класса сливаются в один сегмент (FAQ). Тот же участник и класс с разрывом < 1 с — одно событие.
- Ставим все подходящие классы, перекрытия — норма:** например, `accident` → `stopped_vehicle` (если ≥ 10 с) → `congestion`.
- Событие обрезано клипом — начало 0.0 или конец = длительность. Сомнительные случаи — в `notes` с флагом, решаем арбитражем.
- `near_miss` размечаем исчерпывающе:** их окна бесплатны для Part B (кадры вокруг near-miss игнорируются метрикой).

Класс	start (конвенция команды)	end
<code>accident</code>	первый кадр физического контакта (или потери управления, которая заканчивается ударом)	все участники остановились или покинули кадр; обломки улеглись
<code>near_miss</code>	начало уклонения (резкое торможение или манёвр)	конфликт разрешён: пути разошлись, скорости нормальные
<code>red_light</code>	пёрёд ТС пересекает стоп-линию при красном для его направления	ТС покинуло перекрёсток или кадр
<code>wrong_way</code>	ТС впервые движется против направления полосы / въезжает на встречную	вернулось в правильную полосу или покинуло кадр
<code>illegal_u_turn</code>	начало смены курса внутри зоны запрета разворота	ТС устойчиво едет в обратном направлении
<code>illegal_turn</code>	ТС начинает запрещённую траекторию поворота	поворот завершён, ТС в полосе выезда
<code>solid_line_crossing</code>	первое колесо касается или пересекает сплошную	ТС полностью в другой полосе

Класс	start (конвенция команды)	end
stopped_vehicle	момент остановки на проезжей части (не в очереди у светофора); событие оставляем, только если стоянка ≥ 10 с [opr.]	ТС поехало, убрано или покинуло кадр
stop_line	ТС остановилось за стоп-линией при красном	загорелся зелёный
jaywalking	пешеход ступил на проезжую часть вне перехода	пешеход покинул проезжую часть
failure_to_yield	ТС въезжает на переход, когда на нём пешеход (или он на него ступает)	ТС покинуло переход
congestion	очередь перестала двигаться по всем полосам направления (H1; вариант H2 — отдельным флагом)	поток восстановился
road_obstacle	объект или животное впервые на проезжей части	убран, ушёл или конец клипа
fire_smoke	первый видимый дым или пламя	больше не видно или конец клипа

Спорные случаи помечаем флагом, чтобы оценивать в обоих вариантах: остановка автобуса и такси, парковка, очередь на красный (H1/H2), рабочие и полиция на дороге, человек выходит из машины, пешеходы после аварии, конусы и ограждения, машина за стоп-линией без въезда в перекрёсток.

6.6 Двойная разметка, согласие, арбитраж

- Калибровка (Ч1–1.5).** Все трое размечают одни и те же 60–90 с одного клипа, каждый в своём проекте. Для каждой пары — конвертация и `evaluate.py`, разбор каждого расхождения, заморозка гайдлайна v1.1.
- Двойная разметка (Ч1.5–4).** Каждое видео размечают двое независимо, пары ротируются: A+B, B+C, C+A. Модель не показывается; после слепого прохода можно просмотреть кандидатов детектора, чтобы найти пропуски. Один проход $\approx 4\text{--}6\times$ длительности видео (оценка).
- Согласие (Ч4–5).** Разметка B оценивается как предсказания против A официальным `evaluate.py`, по классам и по каждому τ (F1 симметрична — одного направления достаточно). **Цель — $F1 \geq 0.8$ при $\tau = 0.5$.** Если по классу ниже ≈ 0.7 — правило неоднозначно, сначала чиним гайдлайн.
- Арбитраж.** Третий человек в проекте `gold` импортирует X как аннотацию, Y как prediction и правит. Итог — `dev_gt.json`.
- IoU между аннотаторами для `near_miss` и коротких классов показывает реальный потолок $F1@0.7$; эта статистика идёт в EDA-карточку №16 на сайте.

```
# согласие: A как GT, B как предсказания
python evaluate.py --pred pred_B.json --gt
gt_A.json --per-video
# pred_B.json = {"videos": {"vid": {"events":
gt_B[vid]["events"]}}}
# ("risk" необязателен; без аварий в GT M = Score
A)

# список несовпавших событий в обе стороны
python tools/annotation/disagree.py gt_A.json
gt_B.json 0.5
# A->B C3896.MP4 [62.396, 91.024,
'stopped_vehicle'] best tIoU 0.41
# B->A C3896.MP4 [201.1, 206.3, 'jaywalking'] best
tIoU 0.0
```

Этап	Время (оценка)
Прокси, хост LS, гайдлайн v1, опорные кадры	Ч0–1, параллельно
Калибровка на 60–90 с	Ч1–1.5
Двойная разметка всех сэмплов	Ч1.5–4
Согласие и арбитраж	Ч4–5
Боксы и геометрия	параллельно (C + свободные)
Итого на ≈ 25 мин сэмплов	$\approx 6\text{--}8$ ч wall-clock на троих

Калибровка смещения по dev переносит **наши** конвенции, а не конвенции организаторов: держим её небольшой, формат сверяем с `examples/ground_truth.json`. Одно видео целиком — hold-out для честной оценки.

6.7 Боксы для дообучения детектора

- **Кадры** — из оригинала, имя файла = индекс кадра источника (`f_000270.jpg` — кадр 270 на 9.009 с): ≈ 1 кадр на 1–2 с, плотнее вокруг каждого события, плюс все кадры с препятствиями и дымом. Итого ≈ 300 –600 кадров.
- **Классы:** `car, bus, truck, motorcycle, bicycle, pedestrian, tl_red, tl_yellow, tl_green, tl_off, obstacle, smoke, fire`.
- **Предразметка:** учитель YOLO26x (или YOLO11) на 4K-тайлах; для редкого — промпты YOLOE / Grounding DINO: «debris», «fallen object», «animal», «smoke», «fire», «traffic light».
- **Правка** ≈ 20 –40 с на кадр (оценка), экспорт YOLO + COCO; треки при необходимости — CVAT на прокси.
- **Псевдометки:** уверенные боксы ещё на ≈ 5 –10k кадрах с фильтром временной согласованности; с hold-out видео псевдометки для обучения не брать.

```
# каждый 30-й кадр (~1 кадр/с)
ffmpeg -hide_banner -i C3896.MP4 -map 0:v:0 \
-vf "select='not(mod(n\,30))'" -fps_mode
passthrough \
-frame_pts true -q:v 2 frames/C3896_%06d.jpg
# плотнее внутри окна события: кадры 360..570,
каждый 5-й
ffmpeg -hide_banner -i C3896.MP4 -map 0:v:0 \
-vf
"select='between(n\,360\,570)*not(mod(n\,5))'" \
-fps_mode passthrough -frame_pts true -q:v 2
frames/C3896_%06d.jpg
```

X-AnyLabeling: `uv pip install "x-anylabeling-cvhub[gpu]"` (или `[cpu]`) в **отдельном** venv (тянет `opencv-contrib-python-headless`). Инструмент однопользовательский — папки кадров делим между людьми, экспорты сливаем Dataro. Чат/VLM-модуль — только локальный Ollama, облачные провайдеры не использовать. Video Classifier для событий не подходит: сегменты в нём не перекрываются.

6.8 Геометрия сцены

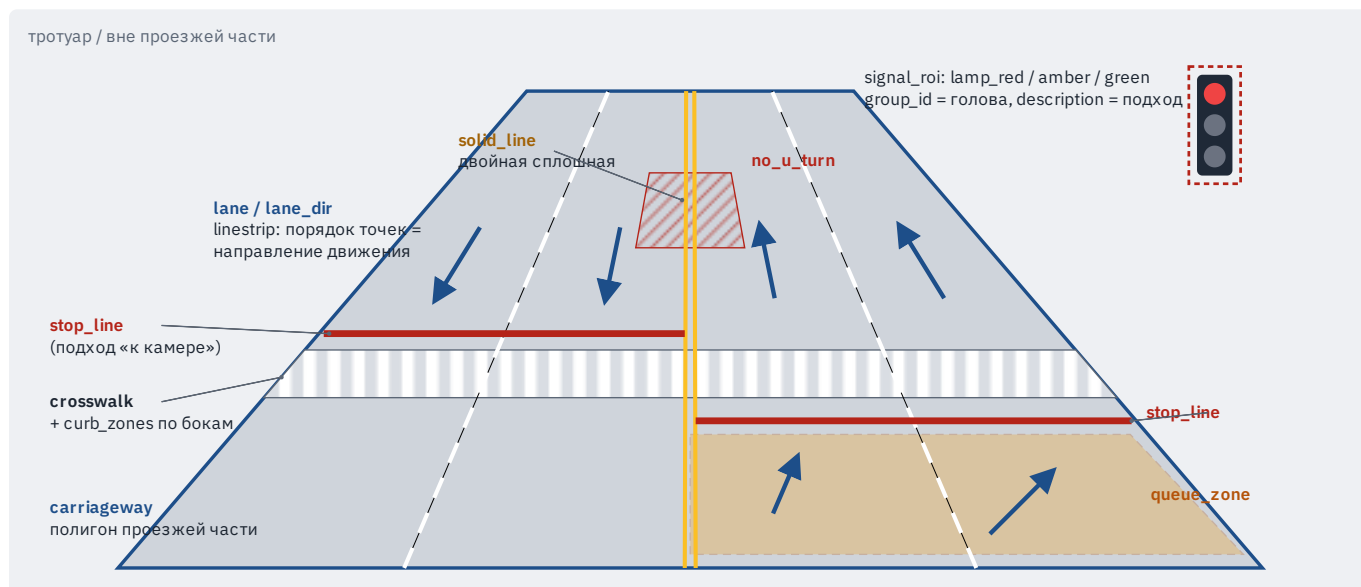


Рис. 8. Условная схема слоёв геометрии на эталонном кадре. Реальная сцена — по `camera.md` (у нас его пока нет [орг.]) и медианному кадру; схема показывает, *какие* объекты рисуются и как называются метки.

- **Опорный кадр** — полноразмерный PNG из оригинала (без `rgb24` из 10-bit может получиться 16-битный PNG):
`ffmpeg -ss 60 -i C3896.MP4 -frames:v 1 -pix_fmt rgb24 ref_C3896.png`. Для `scene.yaml` лучше медианный кадр (медиана раз в 2 с) — на нём нет машин.
- **Метки** (`scene_labels.txt`): `carriageway`, `lane` (`group_id` = номер полосы), `lane_dir` (linestrip, порядок точек = направление), `stop_line`, `crosswalk`, `solid_line`, `signal_roi` или отдельные лампы `lamp_red`, `lamp_amber`, `lamp_green`, `ped_red`, `ped_green`, `no_u_turn`, `turn_restriction`, `queue_zone`.
- Схемы атрибутов у `labelme` нет — команда строго держится соглашения об именах; `--validate-label exact` ловит опечатки.

```
labelme ref_C3896.png --labels scene_labels.txt \
--validate-label exact --output geometry/
python tools/annotation/labelme_to_scene.py \
    geometry/ref_C3896.json scene_C3896.json
# {"carriageway": 1, "lane": 4, "lane_dir": 4,
#  "stop_line": 2, "crosswalk": 1, ...}
```

- Координаты нормируются по ширине и высоте (0–1), прямоугольник `labelme` (2 угла) разворачивается в 4 вершины; скрипт предупреждает, если нет обязательных меток.
- Точки гомографии (≥ 8) кликаются на том же кадре; метрические координаты — из измерений (§4.5).
- **Файл геометрии** — на каждую установку камеры. Проверка: наложить `scene.json` на первый, средний и последний кадр каждого видео, чтобы поймать сдвиг между записями; дальше — автоматическая регистрация.

6.9 Проху-набор аварий, синтетика, гигиена данных

Proху-dev для аварий (в сэмплах аварий, скорее всего, нет)

- 60–100 реальных клипов ACCIDENT с неподвижных CCTV (приоритет — городские перекрёстки сверху), затем TAD и CADP. START берём опубликованный (медиана 3–5 аннотаторов), END размечаем сами.
- Плюс ≥ 2 ч нормального трафика как негативы.
- **Деление по камере или источнику**, а не по клипам.
- Проекция на тест: точность $\approx R \cdot N_{\text{acc}} / (R \cdot N_{\text{acc}} + F_{\text{rate}} \cdot T)$. На сайте честно показываем обе величины.

Синтетические и инъекционные тесты

- Животные, шины, коробки, вклеенные на дорогу на 10–60 с; дым из D-Fire поверх стоящей машины; автобус, перекрывающий стоящую машину.
- Траектории, заданные вручную в метрах: ровно один сегмент с ожидаемыми границами.
- Инъекция столкновений в треки сцены для Part B (§4.9).

Гигиена данных и лицензий

- **Разметка только локально или на своём сервере.** Не загружать футаж организаторов в облачные сервисы: Roboflow Public публикует данные; CVAT Online на free-плане — 1 ГБ и нет командной работы; shared-проекты VIA идут через сервер VGG.
- Модели для предразметки — только помощники; в репозиторий попадают, только если это выбранный детектор, и тогда указываются в README.
- Использование закрытых моделей для предразметки dev — серая зона; считаем «нельзя», пока не ответят [орг.].
- Не собирать записи с той же камеры другими путями. Перед обучением на краулинговых CCTV (ACCIDENT, TAD, CADP) проверить, что ни один клип не показывает место соревнования.
- В README перечислить все датасеты и инструменты, которыми делали обучающие данные.
- На сайте — без распознавания номеров и лиц, без ре-идентификации, без увеличенных кропов; публиковать ли сэмплы целиком — спросить [орг.].

7. Датасеты: ссылки, лицензии, роль

Версии, лицензии и размеры сверены 2026-09-25 по страницам авторов, Kaggle, Hugging Face, arXiv и Zenodo. Метка «(не подтверждено)» — поле проверить не удалось. Источники, данные которых больше нельзя получить за время хакатона, из каталога исключены.

7.1 Правила работы с внешними данными

- **Запрещено:** видео той же камеры или перекрёстка, полученное другими путями. Не скрейпить YouTube, Telegram и городские камеры площадки. Перед обучением на краулинговых CCTV-наборах (ACCIDENT, TAD, CADP, HWID12) проверить, что ни один клип не показывает место соревнования; если показывает — исключить.
- **Время — в секундах.** Наши сэмплы 29.97 fps, датасеты 4–50 fps, PDF говорит про 25 fps: всё пересэмплировать по времени, fps не хардкодить.
- **Лицензии зеркал ненадёжны:** Kaggle- и HF-перезаливы часто подписаны неверно (UA-DETRAC как CC0, TAD как MIT, UCF-Crime как CC0). В README всегда указывать лицензию оригинала.
- **Лучше прикреплять в Kaggle-ноутбуках,** чем качать локально: данные подключаются без скачивания. Обучение — на Kaggle/Colab T4.
- Датасеты в репозиторий не кладём — только скрипты загрузки. Кадры из наборов «только для исследований» на сайте не показываем; в краулинговых CCTV есть номера и лица — размывать в любом публичном демо.

7.2 Основные датасеты (топ-8 + COCO)

Датасет	Домен и разметка	Объём	Лицензия	Для чего нам	Ссылки и загрузка
1. ACCIDENT (CVPR 2026 challenge)	Неподвижные CCTV + синтетика CARLA с CCTV-ракурса. Кадр удара (медиана 3–5 аннотаторов, согласие 0.979 при $\sigma = 1$ с), место, тип (head-on, rear-end, t-bone, sideswipe, single-vehicle, rollover). Синтетика: боксы, маски, ID, траектории. Конец аварии не размечен.	2 027 реальных + 2 211 синт. клипов; 20.16 ГБ; клипы 1–114 с (медиана 26.8), 4–50 fps. До удара медиана 4.67 с контекста	Kaggle: CC BY-NC-SA 4.0; статья: аннотации CC BY 4.0, код Apache-2.0 → применяем строже (NC-SA), цитировать arXiv 2604.09819	accident, Part B (0–5 с до удара), проху-dev	accidentbench.github.io kaggle.com/datasets/picekl/accident github.com/accidentbench/ACCIDENT arxiv.org/abs/2604.09819 kaggle datasets download -d picekl/accident
2. TAD (WSAL, TIP 2021)	Дорожные камеры наблюдения (в основном Китай, часто низкое разрешение). Train — метки уровня видео; подтипы в именах: Accident, IllegalTurn, IllegalOccupation, PedestrianOnRoad, RoadSpills, Normal	500 видео (250 аномальных / 250 нормальных); Kaggle-зеркало 13.44 ГБ (JPG-кадры)	Не указана; research, цитировать TIP 2021 (MIT на Kaggle — заявление загрузчика)	accident, повороты, стоянка, встреча, пешеходы, разливы; Normal — трудные негативы	github.com/ktr-hubrt/WSAL Google Drive (кадры, label.zip) kaggle.com/.../traffic-anomaly-dataset-tad arxiv.org/abs/2008.08944
3. AI City 2021 Track 4	Неподвижные камеры на хайвее, 410p, 30 fps. Train: <video_id> <start> <end> в секундах; аварии и заглохшие ТС	100 train + 150 test видео по ≈15 мин; 15.06 ГБ	Не указана; доступ через Datasets Request Form; research	stopped_vehicle, accident, контроль ложных на длинных видео	aicitychallenge.org/2021-data-and-evaluation .../2021-track4-download .../2021-data-access-instructions после формы: gdown 1gsR4pIEiWK7htPmIjiZjXetaa1X-T-Am
4. UA-DETRAC	Неподвижная приподнятая камера (Китай, 2015). Боксы и треки ТС, ignore-регионы, атрибуты перекрытия и погоды	100 последовательностей, 140k+ кадров 960×540 @25 fps; зеркала 9.9–11.5 ГБ	CC BY-NC-SA 3.0, «academic use only» (метки зеркал неверны)	Дообучение детектора ТС, оценка трекара	sites.google.com/view/daweidu/projects/ua-detrac kaggle.com/datasets/bratjay/ua-detrac-orig kaggle.com/datasets/dtrngc/ua-detrac-dataset arxiv.org/abs/1511.04136
5. D-Fire	В основном уличные и лесные пожары. Боксы fire / smoke: 1 164 только огонь, 5 867 только дым, 4 658 оба, 9 838 негативов	21 527 изображений; Kaggle YOLO-версия ≈3.1 ГБ	CC0-1.0	fire_smoke	github.com/gaia-solutions-on-demand/DFireDataset
6. RAOD (UnicomAI)	Камеры ITS на хайвеях (Китай). Попиксельные маски брошенных на дороге предметов (→ боксы связными компонентами)	557 последовательностей, 18 953 изображения; 5.41 ГБ	Не указана; research; IEEE Access 2024, doi 10.1109/ACCESS.2024.3407955	road_obstacle	github.com/UnicomAI/UnicomBenchmark/.../RAODBenchmark gdown 1WsaBYKtHT55_bdX0JW2vohxsXjFYMdh9 Baidu (код 5tGb)

Датасет	Домен и разметка	Объём	Лицензия	Для чего нам	Ссылки и загрузка
7. MIO-TCD Localization	Тысячи реальных дорожных камер (Канада, США). Боксы 11 классов, включая пешеходов, велосипеды, мотоциклы, фургоны, грузовики	137 743 изображения; 3.74 ГБ прямой загрузки	CC BY-NC-SA 4.0	Детектор (дополняет UA-DETRAC); взять 10–20k изображений	tcd.miovision.com/challenge/dataset.html MIO-TCD-Localization.tar
8. CADP	CCTV с YouTube (оверлеи, дубликаты). 1 416 сегментов аварий, 205 с полной пространственно-временной разметкой; боксы включают людей и двухколёсных	segments.tar.gz 10.3 ГБ + аннотации 568 КБ; в среднем 366 кадров	Некоммерческое исследовательское использование; код GPL-3.0 (не копировать)	accident, Part B (≈3.69 с от начала клипа до аварии)	ankitshah009.github.io/accident_forecasting_traffic_camera Google Drive (главная папка) annotations_CADP.json arxiv.org/abs/1809.05782
COCO 2017	Общие фото; боксы 80 классов (person, bicycle, car, motorcycle, bus, truck, traffic light, dog, horse, cow...)	train2017 19.34 ГБ, аннотации 253 МБ	Аннотации CC BY 4.0; изображения — Flickr-лицензии, много NC/ND, не распространять	Rehearsal против забывания; животные для road_obstacle	cocodataset.org images.cocodataset.org/zips/train2017.zip

Дедуплицировать CADP с ACCIDENT и TAD по YouTube ID. Ссылка «Extracted Frames» у CADP — 404, кадры есть в главной папке. Для `near_miss` публичного CCTV-набора с временной разметкой не нашлось — класс выводим из траекторий.

7.3 Запасные и опциональные

Датасет	Суть	Лицензия	Для чего нам	Ссылки
Street Scene (MERL) запасной	Одна неподвижная камера над улицей; 205 аномальных событий 17 типов (jaywalking 61, biker outside lane 42, car u-turn 5, illegally parked 5...); 202 545 кадров 1280×720 @15 fps; 48.98 ГБ	CC BY-SA 4.0	Валидация правил и калибровка конвенций границ	zenodo.org/records/10870472 merl.com/.../StreetScene
TADBench запасной	Хайвейные CCTV (Китай), метки уровня видео; 4.54 ГБ	Не указана; IEEE Access 2025	accident, кросс-датасетная проверка	github.com/UnicomAI/.../TADBench gdown 14GN1NcWLzN-sbzmzMuSbAg_rZZ5yd26
SO-TAD запасной	CCTV, 2 186 клипов в TAR; ≈27.4 ГБ	Не указана; Neurocomputing 2025	accident, упреждение	huggingface.co/datasets/cccccxy/so-tad
NVIDIA TAR (AI City 2026 Track 3) запасной	Агрегат 8 CCTV-источников; 44 040 псевдо-размеченных QA/CoT, окна MM:SS (1 с), есть и у нормальных клипов; загрузчик одной командой (TAD, TADBench, SO-TAD, HTV, UCF-Crime)	Аннотации CC BY 4.0; видео — лицензии источников	Загрузчик, данные для VLM	huggingface.co/datasets/nvidia/PhysicalAI-Traffic-Anomaly-Reasoning
UCF-Crime (Road Accidents) запасной	1 900 видео, 128 ч; ≈150 видео аварий; временные метки только для теста	Не указана; research (CVPR 2018)	Позитивы и негативы аварий	crcv.ucf.edu/projects/real-world Kaggle-зеркало bypkk/ucf-crimes (читать только Videos/RoadAccidents)
TUMTraF-Accid3nD запасной	Придорожные камеры + LiDAR, хайвей A9, реальные аварии; 111 945 кадров @25 Гц, 2D/3D боксы, треки	CC BY-NC-SA 4.0; dev-kit MIT	Проверка траекторных правил аварий	accident-dataset.github.io arxiv.org/abs/2503.12095
FASDD (CV-подмножество) запасной	Огонь и дым, ≈95к изображений, 3.41 ГБ	CC BY-SA 4.0 (DataCite)	fire_smoke: разнообразие дыма	huggingface.co/datasets/seawsurf/fire_smoke_dataset_fasdd_cv doi.org/10.57760/sciencedb.j00104.00103
Pyro-SDIS опц.	Дым (Pyronear)	Apache-2.0	fire_smoke	huggingface.co/datasets/pyronear/pyro-sdis
MEVA опц.	Наземные камеры, постановочные действия: vehicle_makes_u_turn, turns_left/right, starts/stops/reverses; 328 ч	CC BY 4.0	illegal_u_turn, illegal_turn, stopped_vehicle	mevadadata.org aws s3 ls --no-sign-request s3://mevadadata-public-01/drops-123-r13/
SinD / SinD 2.0 опц.	Дрон, 6 регулируемых перекрёстков: траектории, состояния светофоров, HD-карты; нарушения на красный, встреча, перестроение через сплошную, конфликты	Конфликт: CC0 в LICENSE против бейджа «Non-commercial» → считаем некоммерческим	Настройка и юнит-тесты правил (не для обучения image-моделей)	github.com/SOTIF-AVLab/SinD — заявка по e-mail с учебного адреса, может занять дни
AI City 2021 Track 1 опц.	Перекрёстки, ROI и movement-of-interest на камеру; 31 клип	Не указана; через форму	Шаблон допустимых манёвров	aicitychallenge.org/2021-track1-download

Датасет	Суть	Лицензия	Для чего нам	Ссылки
Open Images V7 опц.	Cattle, Goat, Horse, Dog, Box, Tire, Barrel, Cart... (класса «Traffic cone» нет)	Аннотации CC BY 4.0	Животные и предметы для road_obstacle	storage.googleapis.com/openimages/web (FiftyOne zoo)
Светофоры: S2TLD, LISA, Bosch STL опц.	Кропы светофоров (в основном dashcam)	S2TLD MIT/Apache-2.0; LISA CC BY-NC-SA 4.0; Bosch — некоммерческая	Предобучение классификатора ламп	S2TLD на HF (1.43 ГБ); LISA на Kaggle (5.16 ГБ); Bosch на Zenodo
Highway Traffic Videos опц.	Пробки на хайвее, 92 МБ	CC0 (Kaggle), WSDOT courtesy	congestion	Kaggle
CCD, BDD100K опц.	Dashcam (движущаяся камера)	CCD: MIT (код, аннотации); BDD100K: образование и исследования	Максимум предобучение Part B; светофоры и двухколёсные	github.com/Cogito2012/CarCrashDataset github.com/bdd100k/bdd100k
MOT17 / MOT20 опц.	Треки пешеходов, статичные камеры	Не указана на страницах	Оценка трекера на плотных переходах	motchallenge.net/data/MOT20

7.4 Проверенные и отклонённые

Датасет	Почему не берём
DoTA, DAD, MM-AU, Nexar, A3D	Видеорегистраторы: камера движется, другая точка съёмки. Только общее предобучение, не для финального верификатора (MM-AU — 525.8 ГБ, «only free for academic use»; оригинальный Drive DAD — 401)
Barbados Traffic Analysis (Zindi)	Лицензии нет, правила соревнования запрещают ручную разметку
Objects365	«Available for the academic purpose only», изображения не распространять
CrowdHuman, Cityscapes / CityPersons	Некоммерческие и no-redistribution — модели на них в AGPL-репозиторий не класть
Accident-Bench, TT100K, VisDrone, PIE / JAAD	Нет временных меток для нашей задачи / дорожные знаки / вид с дрона / пешеходы с его-камеры
HWID12, TU-DAT, DFS	Только research/education или без лицензии; при использовании — размывать номера, проверять на кадры площадки

7.5 План загрузки и обучающие смеси

#	Датасет	Размер
1	D-Fire	≈3.1 ГБ
2	MIO-TCO Localization	3.74 ГБ
3	RAOD	5.41 ГБ
4	UA-DETRAC (зеркало)	9.9–11.5 ГБ
5	TAD (Kaggle)	13.44 ГБ
6	AI City 2021 Track 4 (после формы)	15.06 ГБ
7	ACCIDENT (лучше прикрепить в Kaggle)	20.16 ГБ
8	CADP	10.3 ГБ
—	Остальное — только если появится пробел	

Сразу подать заявки: SinD (e-mail, может занять дни) и AI City Challenge Datasets Request Form. В Kaggle сразу прикрепить ACCIDENT, TAD, UA-DETRAC и при необходимости UCF-Crime.

Конвенции времени в датасетах (разные!)

- ACCIDENT: медиана кадра первого контакта; конца нет.
- AIC21-T4: start/end, исходная метрика засчитывает детекцию в пределах 10 с — границы грубее, чем нужно для tIoU 0.7.
- A3D, DoTA: старт — «авария неизбежна», то есть раньше контакта; для настройки границ перерасматривать.
- TAD: train — только уровень видео. NVIDIA TAR: окна MM:SS (1 с). Street Scene: кадры @15 fps.
- Всё переводить в секунды и калибровать смещения начала и конца на нашем dev.

Модель	Обучающая смесь
Основной детектор	≈40% наших кадров (с оверсэмплингом — единственные данные из домена) + ≈40% неподвижных дорожных камер (подмножества UA-DETRAC и MIO-TCO, 10–20k изображений) + ≈20% COCO (люди, двухколёсные, животные, светофоры). С учётом развилки 7.6
<code>fire_smoke</code>	Отдельная маленькая модель на D-Fire (+ FASDD / Pyro-SDIS) + трудные негативы наших видео: стоп-сигналы, красные машины, блики солнца, пыль, выхлоп
<code>road_obstacle</code>	Вычитание фона + детектор на RAOD и животных COCO / Open Images
<code>accident / near_miss</code>	Верификатор на ACCIDENT (реальные + синтетика), TAD, AIC21-T4, CADP, TADBench; пороги калибровать на dev. <code>near_miss</code> — из траекторий
Part B	Позитивные кадры — 0–5 с до удара из ACCIDENT, AIC21-T4, TAD, CADP; остальное — негативы; строго причинно. Dashcam-наборы — максимум предобучение
Классы-правила	Публичные данные только для проверки: подтипы TAD, заглушки TC AIC21-T4, пробки НТВ, нарушения SinD, события Street Scene, манёвры MEVA

7.6 Лицензии и развилка AGPL против NC-SA

Развилка (решить командой до обучения; при сомнении — спросить организаторов; не юридическая консультация)

Ultralytics считает модели, обученные его кодом, AGPL-3.0, а AGPL не допускает дополнительных ограничений вроде «только некоммерческое». Чтобы не смешивать, вариантов два:

1. **(a)** Учить Ultralytics-детектор только на данных без NC: наши кадры, аннотации COCO, D-Fire, Pyro-SDIS, MEVA, Street Scene (SA). NC-наборы (UA-DETRAC, MIO-TCO, ACCIDENT...) использовать для моделей вне Ultralytics (VideoMAEv2 — MIT-код).
2. **(b)** Пермиссивный путь: RF-DETR (Apache-2.0) + `trackers` (Apache-2.0) и явная пометка NC-SA на веса.

SA-наборы (FASDD, Street Scene) требуют share-alike для производных данных; распространяется ли это на веса — юридически не решено. ND / no-redistribution (Cityscapes, CrowdHuman): данные и изменённые копии никогда не публиковать.

Модель / код	Лицензия
Ultralytics YOLO26 / YOLO11 / YOLOE-26 и обученные ими веса	AGPL-3.0 (весь репозиторий под AGPL; код демо-сервера публичный)
RF-DETR N-L, RF-DETR-Seg	Apache-2.0 (XL/2XL — PML 1.0, не брать)
ByteTrack (FoundationVision) / trackers / supervision / SAHI	MIT / Apache-2.0 / MIT / MIT (BoxMOT — AGPL)
X3D (PyTorchVideo) / VideoMAE V2 distilled	Apache-2.0 / код MIT, веса apache-2.0
V-JEPA 2 / 2.1	MIT
Autoware traffic_light_classifier; MM-Grounding-DINO, OWLv2	Apache-2.0
Two-Dimensional-TTC, SSMSOnPlane, ActionFormer, TriDet	MIT
PANNs Cnn14_16k	код MIT, веса CC-BY-4.0 (указать авторство)
Qwen3-VL-2B, InternVL3.5-1B, MiniCPM-V 4.6	Apache-2.0 (только если VLM пройдёт абляцию)
OpenCV / PyAV	Apache-2.0 / BSD-3-Clause (в колёсах GPL-сборка FFmpeg)

Не брать: DEIMv2 (некоммерческая), FireViewer (запрет распространения весов), RF-DETR XL/2XL (PML 1.0), SAM 3 (SAM License, несовместима с AGPL-бандлом), модели на CrowdHuman, VideoMAE v1 и TimeSformer (CC-BY-NC), готовые «аварийные» чекпойнты без лицензии (TimeSouth и др. — только как идея).

Что написать в README: все датасеты с лицензией *оригинала*; датасеты не распространяются, только скрипты загрузки; если используется Ultralytics — код под AGPL-3.0; веса, обученные на NC/SA-данных, наследуют эти условия («for research / non-commercial use, CC BY-NC-SA 4.0-compatible»); что обучено, а что правило; сиды и источники недетерминизма; кто что сделал.

Лицензии инструментов разметки на выходные данные не распространяются: Label Studio — Apache-2.0, X-AnyLabeling и labelme — GPL-3.0 (JSON наш), FFmpeg используем как внешний инструмент и в репозиторий не кладем.

8. План работ

Принцип: сначала рабочая сдача, потом улучшения. Вехи M0 и M4 не пропускаются никогда. Дедлайн в материалах не указан [орг.], поэтому план расписан в днях от D0 (сегодня, 25.09.2026) и сжимается, если времени меньше недели.

8.1 Роли

Участник	Зона ответственности	Доля ценности
P1: восприятие и правила	Детектор и дообучение, трекер и <code>tracker_config</code> , <code>scene.yaml</code> , гомография, регистрация, статичный движок, светофор, модули правил, постобработка	Part A, ≈42% отбора
P2: оценка, аварии, Part В	Инструменты разметки, <code>labels_to_gt.py</code> , цикл <code>evaluate.py</code> и <code>leave-one-video-out</code> , учёт FP в час, <code>accident</code> / <code>near_miss</code> (кинематика, верификатор, звук), Part В (не больше ≈1 человеко-недели)	Качество Score A и Score B
P3: упаковка, сайт, данные	Скачивание сэмплов, прокси, Kaggle-датасеты, <code>requirements</code> и <code>Docker</code> , <code>env.py</code> , CI, <code>download.sh</code> , репетиции, бенчмарки, EDA, сайт, демо, README и отчёт	25% (сайт) + 15% (код) и защита всего M

Размечают все: каждый сэмпл — двое независимо. Роли разметки A/B/C (раздел 6.1) распределяем между P1–P3 по договорённости; роль C (геометрия и боксы) естественно отдать P1.

8.2 Вехи и критерии выхода

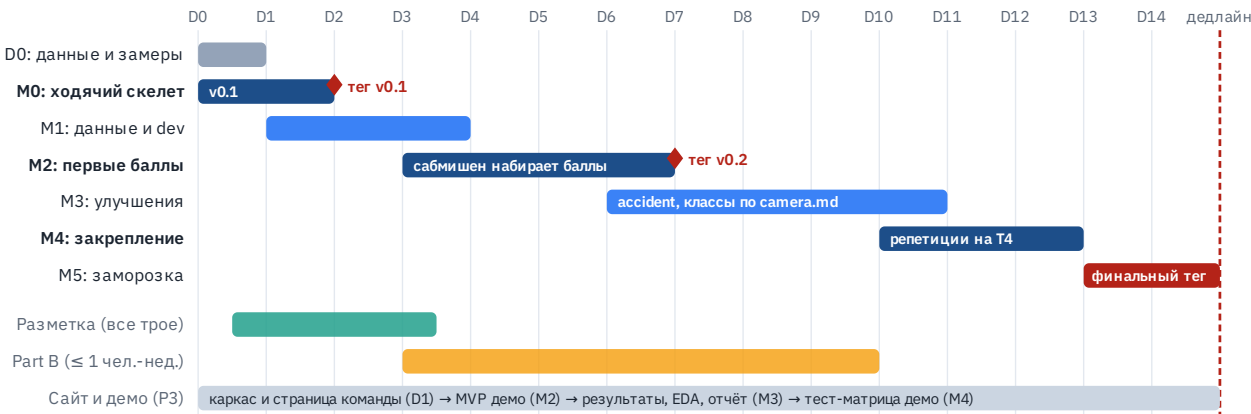


Рис. 9. Вехи при горизонте ≈2 недели. Если до дедлайна меньше недели: M0 в D0, M1 и M2 к D3, M3 в D4–D5, M4 и M5 в D6.

Веха	Содержание	Критерий выхода
D0	P3: копия сэмплов на стороне сервера, приватные Kaggle-датасеты, прокси 1080p/720p с проверкой <code>check_proxy.py</code> , сверка хешей C3897/C3902. P1: <code>ffprobe</code> и аудит контейнера, бенчмарк декодирования на Kaggle T4 (и, если можно, на g4dn.2xlarge spot), <code>budget.py</code> на реальных числах, полный прогон <code>run_submission.py</code> для выбора пина OpenCV. P2: гайдлайн разметки, <code>labels_to_gt.py</code> , прогон <code>evaluate.py</code> . Все: вопросы в канал хакатона	Сэмплы в облаке, таблица стоимостей, вопросы заданы
M0 (D0–D1): ходячий скелет	Структура репозитория, шаблон <code>solution.py</code> , <code>requirements</code> и <code>Docker</code> , <code>env.py</code> , санитайзер, CI. На сайте — каркас и страница команды. Hello-world Space с <code>/ping</code> , решение по хостингу	Чистый клон работает офлайн на Kaggle T4, <code>--validate-only</code> проходит. Тер v0.1
M1 (D1–D3): данные	Двойная разметка и итоговые dev-метки. Кэш треков для всех сэмплов. <code>scene.yaml</code> , гомография, измерение смещений между видео. EDA-карточки 1–4, 7, 8, 11	Согласие аннотаторов посчитано, время на видео измерено
M2 (D3–D6): первые баллы	<code>stopped_vehicle</code> , <code>jaywalking</code> , <code>wrong_way</code> , <code>congestion</code> (H1). Постобработка. Базовый ТТС для Part В. MVP демо: загрузка → шкала событий. Страницы результатов по сэмплам генерируются автоматически	Leave-one-video-out Score A и Score B записаны. Тер v0.2 — сабмишен, который набирает баллы
M3 (D6–D10): улучшения	<code>accident</code> (кинематика + уточнение + верификатор), <code>near_miss</code> при прохождении гейта. Классы по camera.md: <code>red_light</code> , <code>stop_line</code> , <code>failure_to_yield</code> , <code>solid_line_crossing</code> . Уточнение границ, гейтинг классов, калибровка тревог Part В. Абляции, разбор ошибок, галерея, отчёт. Опционально — звук	Каждое изменение проверено A/B на leave-one-video-out, без регрессий по времени и детерминизму
M4 (D10–D12): закрепление	Полная офлайн-репетиция на T4 и в <code>Docker</code> , двойной прогон с <code>diff</code> , fuzz-тесты, <code>predictions_samples.json</code> с кандидата в релиз, README. Тестовая матрица демо, Lighthouse, проверка на телефоне	Матрица репетиций из §4.12 вся зелёная

Веха	Содержание	Критерий выхода
M5 (последние 24–48 ч): заморозка	Только исправления ошибок. Финальный тег. Ссылки на сайте ведут на тег. Space переключён на CPU Upgrade, оповещения включены; теги <code>site-v1</code> и <code>space-v1</code> заморожены за 48 ч	Чистый клон тега воспроизводит <code>predictions_samples.json</code>

Если до дедлайна меньше недели

M0 в D0, M1 и M2 к D3, M3 в D4–D5, M4 и M5 в D6. Из M3 оставляем только `accident` на кинематике и классы, которые прямо поддерживает `camera.md`.
Part B — только TTC-база с консервативным порогом.

Стоп-правила по времени

- Part B — не больше ≈ 1 человеко-недели.
- VLM — не больше 1 дня на абляцию.
- Ночь, ИК и дождь — только после подтверждения организаторов.
- Сайт — ≈ 50 –60 человеко-часов; демо и страница команды — раньше всего остального.

8.3 Ближайшие шаги

- ☒ Анализ задания и стратегия (`WIUT_CV_Track_analysis_RU.md`, 16 разделов, финальная проверка по коду кита — 51 исправление).
- ☒ Каталог инструментов, моделей и датасетов (`WIUT_toolkit_RU.md`); конвертеры `ls_to_gt.py`, `disagree.py`, `labelme_to_scene.py` в `tools/annotation/`.
- ☒ Этот сводный документ.
- ☐ Отправить организаторам вопросы из раздела 10 (главные: формат тестовых видео, CPU оценочной машины, дедлайн, `camera.md`).
- ☐ Скачать сэмплы в обход квоты Drive; сверить хеши C3897 и C3902 (возможен дубликат).
- ☐ Сделать прокси 1080p/720p с сохранением всех кадров и 29.97 fps; проверить `check_proxy.py`.
- ☐ Бенчмарк декодирования на T4 и 8 vCPU; полный `run_submission.py` на реальных сэмплах; выбрать пин OpenCV (4.13.0.92 headless против 4.14.0.94).
- ☐ Решить лицензионную развилку AGPL против NC-SA (раздел 7.6).
- ☐ Тег `v0.1` с рабочим шаблоном; каркас сайта и страница команды.
- ☐ Dev-набор: каждый сэмпл размечают двое, согласие через `evaluate.py`.
- ☐ Реализация: детектор + ByteTrack → следы → правила на геометрии `scene.yaml` → постобработка сегментов.
- ☐ Сайт с live-демо (25% отбора).
- ☐ Перед сдачей убрать или вынести `research/`, чтобы не терять баллы за «no dead code».

8.4 Сайт и live-демо

Сайт даёт 25% отбора: демо 7.5, визуализации сэмплов 5.0, EDA 3.75, подход и отчёт 3.75, команда 2.5, дизайн и extras 2.5 балла. Оценка трудозатрат — 50–60 человеко-часов. Стек: Astro (статика) + ECharts (Apache-2.0) + Mermaid (MIT); без веб-опыта — Quarto.

```
[GitHub Pages: статический сайт]  <- все результаты по сэмплам предрасчитаны;  $\approx 70\%$  рубрики без бэкенда
| (только страница Demo)
+-- @gradio/client (пин версии) -> HF Space A (Gradio 6.28.0), api_name "/analyze", "/ping"
+-- при error/stopped -----> резерв: Modal (или Space B в другом аккаунте)
+-- если всё упало -----> "Попробовать сэмпл" (мгновенно, предрасчитано) + ссылка на UI Space
Загрузки 1–2 ГБ: браузер -> Cloudflare R2 (multipart по 64 МБ, presigned) -> бэкенд берёт по ссылке
```

Хостинг (состояние на сентябрь 2026)

- **HF Space CPU Upgrade** (8 vCPU / 32 ГБ, \$0.03/ч, не засыпает) — лучший выбор на время судейства; с июля 2026 создание Gradio/Docker Space требует PRO (\$9/мес). ZeroGPU бесплатно, но квота считается по запрошенной `duration` и принадлежит посетителю.
- HF T4 не нужен: NVDEC не декодирует 4:2:2, а декод доминирует.
- Modal (\$30/мес кредитов, HTTP-таймаут 150 с → `spawn + poll`) — резерв.
- GitHub Pages: сайт ≤ 1 ГБ, файлы > 100 МиБ блокируются → медиа в R2 или HF dataset repo.
- AGPL-3.0: модель отдаётся по сети, поэтому код демо-сервера публичный, ссылка — на странице демо.
- Пины: Gradio 6.28.0, `@gradio/client` 2.7.0, ECharts фиксированной версии; UptimeRobot + оповещения в общий чат.

Демо: поток

1. Приём: MP4 или MOV (H.264 или HEVC, включая 4K 4:2:2 10-bit), до 2.5 ГБ, анализируются первые 120 с; размер проверяется в браузере до загрузки.
2. Загрузка напрямую в R2 multipart с возобновлением; запасной путь — Gradio `max_file_size="2560mb"`.
3. `ffprobe`: есть видеопоток, ≥ 5 с, 640×360...4096×2304; > 120 с — обрезка с предупреждением.
4. Один проход декода: кадры для анализа + 720p H.264 превью. Тот же `solution.py` с `DEMO_FAST=1`; `RiskEstimator` тоже прогоняется.
5. Прогресс: этап и % кадров с ETA; очередь `max_size=8`, таймаут 8 мин, любое исключение — дружелюбный `gr.Error`.
6. Выход: `events.json` в формате соревнования, `risk.json`, `tracks.json`, превью. Чужая камера — «общий режим» без классов, зависящих от сцены.
7. Три встроенных сэмпла с кэшированными результатами — мгновенный показ.

Плеер результатов (Results, Demo, Dashboard)

- 720p-прокси + canvas-оверлей через `requestVideoFrameCallback`: боксы, хвосты треков, зоны, баннер события.
- Шкала событий (ECharts): строка на класс, GT контуром, предсказание заливкой; TP/FP/FN с переключателем $\tau \in \{0.3, 0.5, 0.7\}$; клик перематывает на `start - 1 с`.
- Кривая риска: линия 0.5, серии тревог со слиянием < 2 с, полосы `[s-5, s)` и `[s-10, s)`, штриховка игнора; тревоги с игнорируемым началом — как отброшенные.
- Таблица событий; горячие клавиши (пробел, J/L, ←/→). Галерея по классам и ≥ 4 разобранные ошибки.

#	EDA-карточка «находка → решение»
1	Аудит контейнера → база времени <code>idx/fps</code> , стратегия декода
2	Стабильность камеры (фазовая корреляция, гироданные <code>rtmd</code>) → нужна ли H(t)
3	Медианные кадры полудня и вечера → эталоны, <code>scene.yaml</code>
4	Освещение и автоэкспозиция → относительные признаки ламп
5–6	Объекты по классам во времени; размер от строки → вход и тайл
7–9	Тепловая карта, поле направлений , OD-матрица → маска дороги, <code>wrong_way</code> , манёвры
10–11	Скорость по полосам, карта времени стоянки → пороги, очередь против <code>stopped_vehicle</code>
12–13	Пешеходы и переходы; фаза светофора и мерцание
14–15	Абляция stride ; распределение TTC → выбор stride, бюджет тревог
16–17	IoU между аннотаторами → min-dur, потолок $\tau = 0.7$; звук → включать ли модуль

Отчёт на одну страницу: TL;DR с цифрами dev, конвейер, что сработало (с дельтами абляций), что не сработало (с цифрами), режимы отказа, время против 3×, что дальше, воспроизводимость. Честно указать: метки наши, выборка мала.

9. Риски

#	Риск	Вероятность	Последствие	Как снижаем
1	Тест в формате 4K 4:2:2 10-bit, а CPU — уровня 4 физических ядер: один декод харнесса ≈1.8х, бюджет превышен	Высокая, если тест в исходном формате	Видео пустое, пропадает и Part A	Вопрос организаторам сразу; бюджет по T _A ; уровни T0–T2; бенчмарк на g4dn; Linux-колесо OpenCV; никаких повторных чтений
2	Установка падает: CUDA/драйвер, нет колеса под их Python, libGL, компиляция	Средняя	Фатально	cu126, headless OpenCV, маркеры для 3.10, constraints, проверка в python:3.10-slim / 3.12-slim, Docker как запасной путь
3	Скрытый выход в интернет: веса по имени, шрифт, lap, CLIP для YOLOE, HF	Средняя	Падение или зависание	env.py, абсолютные пути, пин lap, netguard, --network none
4	Исключение, NaN или субмиллисекундное событие	Средняя	Видео пустое или весь файл отвергнут	Этапы в try/except, санитайзер, fuzz-тесты, --validate-only в CI
5	Segfault, OOM-kill, «липкая» ошибка CUDA, зависание	Низкая–средняя	Весь прогон потерян	Проверка здоровья CUDA, ограничение памяти, подпроцесс для рискованных этапов, свой watchdog
6	Недетерминизм	Средняя	Минус в Reproducibility, возможно нарушение правил	Флаги, сиды, сортировка, никаких решений по часам, двойной прогон с diff
7	Предсказан класс, которого нет в тесте	Высокая	−1/(C +1) от Score A за каждый	Гейтинг p·f > (1−p)·q·S̃, выключатели по классам
8	Границы не совпадают с конвенциями (промахи при τ = 0.7)	Высокая	Заметная часть Score A	Гайдлайн повторяет конвенции, геометрические опоры, перенос начала назад, уточнение на полной частоте
9	Переобучение на 4 сэмплах	Высокая	Слабый результат на тесте	Leave-one-video-out, физические параметры, синтетические тесты
10	Сдвиг камеры между сессиями	Высокая	Ложные срабатывания геометрических классов	Регистрация под каждое видео, отключение хрупких классов при провале
11	Нарушение причинности Part B	Низкая	Дисквалификация	Изолированный модуль, тест причинности в CI, никаких данных из Part A
12	Скачивание весов падает или превышен лимит 5 ГБ	Низкая–средняя	Фатально	Веса в git и Release, SHA-256, зеркало, без LFS, VLM по умолчанию нет
13	Демо спит, упало или отвергло загрузку жюри на 2 ГБ	Средняя	До 7.5 балла	CPU Upgrade, загрузка в R2, обрезка, сэмплы в кэше, резервный бэкэнд, тестовая матрица
14	Сэмплы не скачиваются из-за квоты Drive, мало места на диске	Уже случилось	Потеря дней	Копирование на стороне сервера (Colab, rclone), приватные Kaggle-датасеты, работа на прокси
15	Неверная база времени (25 fps или PTS)	Средняя	Сдвиг всех границ	Везде idx/CAP_PROP_FPS, проверка прокси, frameRate в Label Studio
16	Низкое солнце, тени, автоэкспозиция на закате	Средняя–высокая	Ложные блобы, промахи по лампам	Относительные признаки, защита от глобальных изменений, эталонный кадр для вечера

Соблюдение правил (чек-лист)

- ☐ Только open-weights, без платных API на инференсе (для кода, сайта и отчёта — можно; для предразметки dev считаем «нельзя» до ответа [орг.]).
- ☐ run_submission.py и evaluate.py не менять; в CI сверять хэш со стартовым китом.
- ☐ RiskEstimator.step получает только свои кадры: файл не открывается, выходы Part A не читаются.
- ☐ Два прогона на одной машине дают одинаковый результат.
- ☐ Не собирать записи с той же камеры другими способами.
- ☐ Внешний код и данные атрибутированы в README; готовыми решениями с другими командами не обмениваемся.
- ☐ На сайте только собственные материалы и материалы, на которые есть права; никаких персональных данных сверх видимого в видео.
- ☐ Оценивается тегированный коммит на момент дедлайна.

10. Вопросы организаторам

Вопросы, на которые уже отвечает код харнесса, убраны. Вопрос о передаче выходов Part A в Part B не задаём: PDF прямо это запрещает.

Формат и железо (задать первыми)

1. Тестовые видео — исходные файлы камеры, как сэмплы (3840×2160, 29.97 fps, H.264 High 4:2:2 10-bit, ~140 Мбит/с, со звуком)? Или они перекодированы? (В PDF — «several minutes long at 25 fps», пример GT с fps 25.0, но там же сказано, что сэмплы имеют то же разрешение и частоту, что тест.)
2. Какая точно машина: модель CPU, физические ядра или vCPU (это AWS g4dn.2xlarge?), ОС, версия Python, версия драйвера NVIDIA? Ставится ли всё в чистый `venv`? Есть ли Docker с `nvidia-container-toolkit` и какой будет команда `docker run`?
3. Замеряли ли вы на этой машине собственный цикл `cv2.read()` из `run_submission.py` на таких файлах? У нас на 8 vCPU он один занимает 0.85–1.0× длительности, на 4-ядерном Xeон может дойти почти до 2×. Реально ли уложиться в 3×?
4. Если `RiskEstimator` выбросит исключение (чтобы уложиться в бюджет), засчитываются ли события Part A этого видео (так делает `run_submission.py`) или всё видео считается пустым (так звучит PDF)?
5. Есть ли общий лимит времени на импорт `solution.py` и загрузку моделей до первого видео?
6. `weights/download.sh` запускается на самой оценочной машине? Можно ли там собрать TensorRT-engine или сделать другую разовую подготовку?
7. Финальная ли версия `wiut_cv_scripts.zip` лежит на Drive? Изменится ли железо до дедлайна? **Когда дедлайн** (дата, время, часовой пояс)? **Где файл `camera.md`** — в ките его нет?

Тестовый набор

8. Тестовые видео сняты в те же сессии и с той же установкой штатива, что сэмплы (18.09: C3896/C3897 днём, C3902/C3905 вечером)? Или в другие дни, с переставленной или перезумленной камерой?
9. Сколько примерно тестовых видео и какова общая длительность? Это полные клипы камеры или фрагменты? Есть ли съёмка после заката, ночью, в дождь?
10. Сохраняется ли в тестовых файлах звуковая дорожка и можно ли использовать звук внутри `detect_events` (только Part A)?
11. Как считается `duration` в скрытом GT: как `CAP_PROP_FRAME_COUNT/fps` (так делает харнесс) или по контейнеру?

Конвенции разметки

12. Сливаются ли близкие, но не одновременные события одного класса? С какой точностью ставятся границы (в примере GT — целые секунды)? Есть ли минимальная длительность события?
13. `congestion`: размечается ли обычная очередь на красный? Минимальная длительность? Относится ли правило «одновременные события — один сегмент» к пробке в двух направлениях?
14. `stopped_vehicle`: считаются ли автобусы на остановке, такси при высадке, машины в парковочном ряду? Размечается ли машина, оставшаяся после аварии? Начало — в момент остановки (до истечения 10 с)?
15. Машина пересекла стоп-линию на красный и остановилась до перекрёстка: только `stop_line` или также `red_light`? Как считаются красный+жёлтый и поворот направо по знаку 5.42?
16. Разворот или поворот через сплошную размечается двумя метками (с `solid_line_crossing`)?
17. `jaywalking`: считаются ли люди, выходящие из машины, полицейские, дорожные рабочие? `failure_to_yield`: считается ли случай, когда пешеход идёт на свой красный, а машина едет на зелёный?
18. `road_obstacle`: считаются ли конусы и ограждения, поставленные намеренно, и животное, переходящее дорогу?
19. `accident`: END — только «все остановились или уехали» или включаются последствия? Водители, вышедшие на дорогу после аварии, — `jaywalking`? `near_miss` включает конфликты машина-пешеход и размечен исчерпывающе?

Правила и сайт

20. Допустим ли код под AGPL-3.0 (Ultralytics)? Можно ли использовать закрытую модель только для помощи в разметке dev-набора?
21. Можно ли публиковать сэмплы целиком (с нашей разметкой) на публичном сайте? Можно ли публиковать наши dev-метки?
22. Каким будет тестовый файл жюри для демо (размер, формат, длительность, эта камера или любое видео)? Устроит ли хост, который просыпается примерно за минуту?
23. Доступны ли на запись каталог репозитория, `$HOME` и `/tmp`? Все 8 ядер выделены нам?

11. Ссылки

Ссылки на датасеты с назначением и лицензиями — в разделе 7; ссылки на сэмплы — в разделе 3.1. Ниже — всё остальное.

Инструменты разметки и утилиты

- Label Studio: github.com/HumanSignal/label-studio · labelstud.io/tags/timelinelabels · labelstud.io/tags/video · ML-бэкенды: github.com/HumanSignal/label-studio-ml-backend
- X-AnyLabeling: github.com/CVHub520/X-AnyLabeling
- labelme: github.com/wkentaro/labelme
- CVAT: github.com/cvat-ai/cvat · docs.cvat.ai
- ELAN: archive.mpi.nl/tla/elan · BORIS: boris.unito.it
- VIA 3: robots.ox.ac.uk/~vgg/software/via
- FFmpeg: ffmpeg.org · drawtext: ffmpeg.org/ffmpeg-filters.html#drawtext-1 · mpv: mpv.io/installation
- uv: github.com/astral-sh/uv · FiftyOne: github.com/voxel51/fiftyone · Datumaro: github.com/open-edge-platform/datumaro
- opencv-python: pypi.org/project/opencv-python · миграция 4→5: github.com/opencv/opencv/wiki/OpenCV-4-to-5-migration · PyAV: pypi.org/project/av
- rclone Drive: rclone.org/drive · Kaggle datasets: kaggle.com/docs/datasets

Детекторы, трекеры, восприятие

- YOLO26: docs.ultralytics.com/models/yolo26 · arxiv.org/abs/2606.03748 · YOLO11: docs.ultralytics.com/models/yolo11 · YOLOE: docs.ultralytics.com/models/yoloe · трекинг: docs.ultralytics.com/modes/track · лицензия: ultralytics.com/license
- Beca: github.com/ultralytics/assets/releases/download/v8.4.0/yolo26m.pt (и [yolo26s.pt](#), [yolo26m-seg.pt](#), [yoloe-26l-seg.pt](#) в том же релизе)
- RF-DETR: github.com/roboflow/rf-detr · arxiv.org/abs/2511.09554 · D-FINE: github.com/Peterande/D-FINE · RT-DETRv4: github.com/RT-DETRs/RT-DETRv4
- ByteTrack: github.com/FoundationVision/ByteTrack · Roboflow trackers: github.com/roboflow/trackers · OC-SORT: github.com/noahcao/OC_SORT · BoxMOT: github.com/mikel-brostrom/boxmot
- Autoware traffic_light_classifier: huggingface.co/AutowareFoundation/traffic_light_classifier
- Огонь/дым (день 1): huggingface.co/rabahdev/fire-smoke-yolov8n
- MM-Grounding-DINO tiny: huggingface.co/openmmlab-community/mm_grounding_dino_tiny_o365v1_goldg_v3det · OWLv2: google.github.io/owlv2-base-patch16-ensemble
- SAM 2: github.com/facebookresearch/sam2 · SAHI: arxiv.org/abs/2202.06934
- NVDEC-матрица: developer.nvidia.com/video-encode-decode-support-matrix · совместимость TensorRT engine: docs.nvidia.com/.../engine-compatibility.html
- PyTorch cu126: download.pytorch.org/whl/cu126 · воспроизводимость: docs.pytorch.org/docs/2.14/notes/randomness.html · RFC по CUDA: github.com/pytorch/pytorch/issues/190385

Модели событий, VLM, TTC

- VideoMAE V2: huggingface.co/OpenGVLab/VideoMAE2 · github.com/OpenGVLab/VideoMAEv2 · MODEL_ZOO: .../docs/MODEL_ZOO.md
- PyTorchVideo (X3D): github.com/facebookresearch/pytorchvideo · model zoo: pytorchvideo.readthedocs.io/en/latest/model_zoo.html
- V-JEPA 2 / 2.1: github.com/facebookresearch/vjepa2 · MoViNet: huggingface.co/kfkas/movinet-a0-stream-pytorch
- ActionFormer: github.com/happyharrycn/actionformer_release · TriDet: github.com/dingfengshi/TriDet · OpenTAD: github.com/sming256/OpenTAD
- Qwen3-VL-2B: huggingface.co/Qwen/Qwen3-VL-2B-Instruct · Cosmos-Reason2-2B: huggingface.co/nvidia/Cosmos-Reason2-2B · MiniCPM-V 4.6: huggingface.co/openbmb/MiniCPM-V-4.6
- Two-Dimensional-TTC: github.com/Yiru-Jiao/Two-Dimensional-Time-To-Collision (SSMsOnPlane — репо того же автора) · Emergency Index: github.com/AutoChengh/EmergencyIndex
- FHWA SSAM: highways.dot.gov/.../FHWA-HRT-08-049.pdf · MTTC (Ozbay et al.): DOI 10.3141/2083-12 · Saunier & Sayed: DOI 10.3141/2083-11
- PANNs: github.com/qiuqiangkong/audioset_tagging_cnn · zenodo.org/records/3987831

Статьи, на которые опирается план

- ACCIDENT: arxiv.org/abs/2604.09819 · SynCrash: arxiv.org/abs/2608.29759 · TimeSouth: arxiv.org/abs/2606.09542
- Qwen3-VL-32B + YOLO11x + BoT-SORT на реальных CCTV: arxiv.org/abs/2608.08867 · zero-shot CLIP-пайплайн: arxiv.org/abs/2604.09685
- Lipton, Elkan, Narayanaswamy — пороги для F1: arxiv.org/abs/1402.1892 · sklearn AP: scikit-learn.org/.../average_precision_score.html
- UString: arxiv.org/abs/2008.00334 · FastTracker: arxiv.org/abs/2508.14370 · 1-euro filter: gery.casiez.net/publications/CHI2012-casiez.pdf

ПДД и нормы Узбекистана

- uzpdd.uz/pddtext.php?id=7 · en.wikipedia.org/wiki/Road_signs_in_Uzbekistan · lex.uz/ru/docs/4983146 (O'z DSt 3419:2019) · lex.uz/docs/4831939 (ЗПУ-547)

Хостинг и веб

- HF Spaces: huggingface.co/docs/hub/spaces-overview · ZeroGPU: huggingface.co/docs/hub/spaces-zerogpu · цены: huggingface.co/pricing
- Modal: modal.com/pricing · Cloudflare R2: developers.cloudflare.com/r2/platform/limits · presigned: .../r2/api/s3/presigned-urls
- GitHub Pages: docs.github.com/.../github-pages-limits · Gradio JS client: gradio.app/docs/js-client
- requestVideoFrameCallback: developer.mozilla.org/.../requestVideoFrameCallback · UptimeRobot: uptimerobot.com/pricing